# Entwicklung eines Interpreters für die Sprache SetIX

## Projektarbeit

im Studiengang Informatik Cyber Security

an der Dualen Hochschule Baden-Württemberg Mannheim

von

**Brandtner, Maximilian Immanuel**

Abgabedatum: 15. April 2026

| | |
|---|---|
| Bearbeitungszeitraum | 01.10.2025 – 15.04.2026 |
| Kurs | TINF23CS1 |
| Gutachter | Prof. Dr. Karl Stroetmann |
| Unterschrift Gutachter | _____ |

## Erklärung

gemäß § 35 (1) der „Studien- und Prüfungsordnung DHBW Technik" vom 7. März 2024.

Ich habe die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet.

Wir versichern zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.*

* falls beide Fassungen gefordert sind

Ehningen, 8. April 2026

Ort, Datum Unterschrift

**Abstract**

SetlX is an imperative programming language designed to closely mirror the notation of mathematics and set theory, drawing inspiration from SETL, one of the first languages to offer sets and maps as first-class data types. This makes SetlX a natural fit for teaching and prototyping algorithms in an academic context. The reference implementation is an interpreter written in Java, using ANTLR4 as its parser generator. This project reimplements SetlX as an interpreter in Rust, transforming the ANTLR4-based ALL(*) grammar into a LALR(1) grammar using LALRPOP, and constructing an appropriate interpreter pipeline.

**Abstract**

SetlX ist eine imperative Programmiersprache, die darauf ausgelegt ist, die Notation der Mathematik und Mengenlehre möglichst direkt abzubilden. Sie lehnt sich an SETL an, eine der ersten Programmiersprachen, die Mengen und Abbildungen als erstklassige Datentypen unterstützt. Dies macht SetlX zu einem natürlichen Werkzeug für die Vermittlung und Entwicklung von Algorithmen im akademischen Kontext. Die Referenzimplementierung ist ein in Java geschriebener Interpreter, der ANTLR4 als Parser-Generator verwendet. Dieses Projekt reimplementiert SetlX als Interpreter in Rust, transformiert die ANTLR4-basierte ALL(*)-Grammatik in eine LALR(1)-Grammatik unter Verwendung von LALRPOP und konstruiert eine passende Interpreter-Pipeline.

# Contents


**1. Introduction** **1**

1.1. Motivation . . . . . 1

1.2. Problem Statement . . . . . 1

1.3. Relevance . . . . . 1

1.4. Objective . . . . . 2

**2. Background** **3**

2.1. Concrete Syntax Tree (CST) Construction . . . . . 3

2.1.1. Tokenizing . . . . . 3

2.1.2. Parsing . . . . . 3

2.2. Intermediate Representations . . . . . 4

2.2.1. Three-Address Code Form . . . . . 4

2.2.2. Control Flow Graphs and Data Flow Graphs . . . . . 5

2.2.3. Static Single Assignment Form (SSA) . . . . . 6

2.2.4. Value Propagation . . . . . 7

2.2.5. Dead Code Elimination . . . . . 8

2.2.6. Interprocedural Optimizations . . . . . 9

2.2.7. Register Allocation . . . . . 10

2.3. Memory Management . . . . . 11

2.3.1. Reference Counting . . . . . 11

2.3.2. Garbage Collection . . . . . 11

2.3.3. Resource Acquisition Is Initialization (RAII) . . . . . 12

2.4. Non-local Control Flow and Exception Handling . . . . . 13

2.4.1. Two-phase unwinding . . . . . 14

2.4.2. Landing pads . . . . . 14

2.5. Interpreter Models . . . . . 15

2.5.1. Tree-Walking Interpreters . . . . . 15

2.5.2. Bytecode Virtual Machines . . . . . 15

**3. Implementation** **17**
3.1. Parser and Passes over the Concrete Syntax Tree (CST) . . . . . 17
3.1.1. Reduce-Reduce Conflicts . . . . . 17
3.1.2. Further Constraints Imposed by LALRPOP . . . . . 20
3.1.3. CST Passes . . . . . 20
3.2. Internal Interpretation Concepts . . . . . 21
3.2.1. Control Flow . . . . . 21
3.2.2. Heap Memory Management . . . . . 23
3.2.3. Stack . . . . . 26
3.2.4. Calling Conventions . . . . . 27
3.2.5. Exception Handling . . . . . 27
3.2.6. Classes . . . . . 28
3.3. Interpretation . . . . . 29
3.3.1. Value Representation . . . . . 30
3.3.2. Predefined and Builtin Procedures . . . . . 31

**4. Conclusion** **32**
4.1. Summary and Reflection . . . . . 32
4.2. Outlook . . . . . 33
4.2.1. Semantic Correctness . . . . . 33
4.2.2. Completeness . . . . . 33
4.2.3. Performance . . . . . 34

**A. Illustrative Examples** **35**
A.1. Reference Counting in jq . . . . . 35
A.2. Procedure Intermediate Representation (IR) Example . . . . . 36
A.3. Heap Mutable Borrowing . . . . . 38
A.4. Stack Closure Example . . . . . 39
A.5. Class IR Example . . . . . 40

**B. Supported Features** **42**
B.1. Predefined Procedure Support . . . . . 42

**C. Test Results** **48**

**D. LALR(1) SetIX Grammar** **59**

# Acronyms

**ABI** Application Binary Interface

**ALL** Adaptive LL(*)

**AST** Abstract Syntax Tree

**BNF** Backus-Naur Form

**CFG** Control Flow Graph

**COW** Copy-On-Write

**CST** Concrete Syntax Tree

**DAG** Directed Acyclic Graph

**DCE** Dead-Code Elimination

**DFA** Deterministic Finite Automaton

**DFG** data flow graph

**DSL** Domain Specific Language

**EBNF** Extended Backus-Naur Form

**HIR** High-Level Intermediate Representation

**IR** Intermediate Representation

**ISA** Instruction Set Architecture

**JIT** Just In Time Compilation

**LICM** Loop-Invariant Code Motion

**LSRA** Linear Scan Register Allocation

**MIR** Mid-Level Intermediate Representation

**NFA** Non-Deterministic Finite Automaton

**PRE** Partial Redundancy Elimination

**RAII** Resource Acquisition Is Initialization

**SCBP** Second-Chance Bin Packing

**SSA** Static Single Assignment Form

**VM** Virtual Machine

# List of Figures

2.1. Three-address code for a nested arithmetic expression . . . . . 5
2.2. Non-Convergence in Value Range Propagation . . . . . 8

3.1. Heap Ownership IR Example . . . . . 25

A.1. jq example expression . . . . . 35
A.2. jq reference counting example . . . . . 36

# List of Tables


2.1. Source code to three-address code . . . . . 5
2.2. Three-address code to SSA . . . . . 6
2.3. Copy Propagation Example . . . . . 7

3.1. Lambda parameter rules: ANTLR4 EBNF vs. LALRPOP EBNF . . . . . 18
3.2. Assignment rules: ANTLR4 EBNF vs. LALRPOP EBNF . . . . . 19
3.3. CST Passes . . . . . 21
3.4. Control Flow Terminal Statements . . . . . 22
3.5. Lowering While Statement IR Example . . . . . 23

A.1. Procedure IR example . . . . . 37
A.2. Heap Memory Management IR Example . . . . . 38
A.3. Stack Closure IR Example . . . . . 39
A.4. Class IR Example . . . . . 40

B.1. Predefined Procedure Support . . . . . 42

C.1. SetlX Logic Lecture Test Results . . . . . 48
C.2. SetlX Interpreter Test Results . . . . . 50

# 1. Introduction

## 1.1. Motivation

SetlX [21] is a programming language designed to closely mirror the notation of mathematics and set theory. It draws inspiration from SETL [19], one of the first programming languages to offer sets and maps as first-class data types. As SetlX extends the concepts of SETL with a syntax that closely mimics set-theoretic and mathematical notations, the gap between the formal specification and a working implementation is reduced greatly. Thus, the language is ideal to teach and prototype algorithms in an academic context. This project explores the design of the language, by reimplementing it as an interpreter in the Rust programming language.

## 1.2. Problem Statement

The reference implementation of SetlX is an interpreter written in Java, using ANTLR4 [16] as its parser generator. ANTLR4 uses the ALL(*) parsing strategy, a powerful but complex formalism that cannot be ported directly to other parser generators without introducing grammatical ambiguities. This project reimplements SetlX from the ground up in Rust, transforming the ANTLR4-based grammar into one expressed in LR(1) using LALRPOP [15]. This shift requires a careful examination of the grammar, resolving ambiguities that ALL(*) handles implicitly, and producing an explicit, unambiguous grammar that satisfies the stricter requirements of a bottom-up parser.

## 1.3. Relevance

Reimplementing an existing language offers a concrete and well-scoped opportunity to engage with foundational topics in compiler and interpreter design. This project touches on several such topics: grammar engineering, the practical differences between parsing formalisms, and the construction of an interpreter pipeline in a systems programming language. The transition from ALL(*) to LR(1) in particular surfaces the tradeoffs between parser expressiveness and

the constraints that more restricted formalisms impose. Rust, with its expressive type system and emphasis on correctness, is a natural fit for a project of this kind. The aim is not to improve SetlX as a tool, but to use its reimplementation as a way to study these topics in a practical setting.

## 1.4. Objective

This project aims to produce a working interpreter for SetlX in Rust [24], with semantic equivalence to the reference implementation as its primary goal. A central part of this work is the transformation of the SetlX grammar from its original ANTLR4 form into a LR(1) grammar suitable for use with LALRPOP. The design prioritises clarity and maintainability of the implementation over raw performance, reflecting the academic nature of the project.

# 2. Background

The following sections survey the conceptual foundations that underlie both the reference implementation and this project; not all are necessary to follow the conceptual implementation design described in Chapter 3, but they inform the design decisions made throughout.

In interpreters, the front-end produces an Abstract Syntax Tree (AST) or an Intermediate Representation (IR), which is executed directly rather than compiled into machine code. During execution, the interpreter may perform optimizations such as constant folding or expression simplification to improve performance.

## 2.1. Concrete Syntax Tree (CST) Construction

### 2.1.1. Tokenizing

Before a source file can be parsed into a CST, it is first tokenized by a lexer[1]. The lexer converts the raw input into a sequence of tokens, each representing a basic syntactic element such as a keyword, identifier, or operator. Lexers often define tokens using regular expressions. If the lexer encounters input that does not match any token, it reports a lexical error. Most parsers operate on token sequences rather than raw text to simplify parsing, although scannerless parsers exist as well [25].

### 2.1.2. Parsing

The parser takes the token list from the lexer and constructs a CST based on the language's formal grammar. Formal grammars define the syntactic rules of a language and are usually classified as either regular or context-free [25]. Regular grammars restrict rules to derive a terminal optionally followed by a nonterminal, while context-free grammars allow more general derivations [6, 7].

Parsers are broadly categorized as top-down or bottom-up. Top-down parsers, such as recursive descent parsers, recursively expand nonterminals. These parsers often require back-

[1] A lexer may also be called a scanner or tokenizer.

tracking or lookahead. Bottom-up parsers, like LR, SLR, and LALR parsers, use a stack and an action table to shift tokens and reduce sequences according to grammar rules [10].

#### LR Parsing

LR parsers build the parse tree from the leaves (input tokens) up to the root. The action and goto tables encode a Deterministic Finite Automaton (DFA) that guides parsing. Actions include shift, reduce, accept, or error. Shifting pushes a token and its state onto the stack; reducing replaces a sequence of states with a nonterminal according to a grammar rule. A semantic stack mirrors this process to construct a CST, which may later be transformed into an AST for further processing [23].

#### ALL(*) Parsing

While LR(1) parsers are efficient for deterministic grammars, they are limited to a fixed one-token lookahead and bottom-up construction. ALL(*) parsing — which is used by ANTLR4 — by contrast, uses a top-down strategy with adaptive lookahead. It constructs an Non-Deterministic Finite Automaton (NFA) to explore multiple parsing paths and select grammar rules dynamically, allowing it to handle a wider range of grammars, including left-recursive and ambiguous constructs.

## 2.2. Intermediate Representations

An optimizing interpreter performs various optimizations on the AST it receives to generate performant code. Different optimizations benefit from an intermediate representation which is represented in different formats.

### 2.2.1. Three-Address Code Form

Three-address code represents a program as a sequence of simple instructions. It is based on the concepts of *addresses* and *instructions*. An address can be a variable name, a constant, or a compiler-generated temporary, which holds intermediate results. Instructions can include binary or unary assignments, conditional or unconditional jumps, and procedure calls [1]. For instance, the expression $(a + (a \cdot (b - c))) + ((b - c) \cdot d)$ can be represented in three-address code as follows:

```
t_1 = b - c
t_2 = a * t_1
t_3 = a + t_2
t_4 = t_1 * d
t_5 = t_3 + t_4
```

Figure 2.1.: Three-address code for a nested arithmetic expression

Three-address code does not encode high-level constructs such as loops or iterators directly. These constructs are instead represented using `goto` statements. Instructions are often organized into basic blocks, which are sequences of instructions starting with a label and ending with a terminal statement such as a jump, or branch. In an optimizable three-address code IR, branch instructions explicitly define both `if` and `else` targets. While the serialized representation of instructions may appear linear, basic blocks are internally stored as a graph to capture control flow [1].

For example, a `while` loop can be encoded as a conditional check that jumps to the end of the loop if the condition fails, the loop body instructions, and a `goto` statement back to the condition check:

Table 2.1.: Source code to three-address code

| | |
|---|---|
| `while(a < 4) {`<br>`  a++;`<br>`}` | `1:`<br>`  t_1 = a < 4`<br>`  if t_1`<br>`  goto 2`<br>`  else`<br>`  goto 3`<br>`2:`<br>`  a = a + 1`<br>`  goto 1`<br>`3: /* end of loop */` |

Three-address code does not require a specific data structure for storing instructions; basic blocks are commonly stored as arrays or linked lists. It is widely used as an intermediate representation because it simplifies analysis and reduces the number of special cases in compiler and interpreter implementations. While high-level constructs like loops can be exploited for language-specific optimizations, they are generally unnecessary for language-independent optimizations and often add complexity.

### 2.2.2. Control Flow Graphs and Data Flow Graphs

Many optimizations rely on understanding the logical control flow within code. A Control Flow Graph (CFG) encodes such information. It works by dividing the IR into basic-blocks. Every

basic-block is a node in the CFG. If a basic-block $a$ can be reached from a basic block $b$ through a jump instruction, an edge is added in the CFG between the corresponding two nodes [4].

A data flow graph (DFG) encodes the flow of data within the code. It stores which instructions access or modify a given variable. In a DFG each node represents a data element, such as variables, constants, and operations. Edges within the DFG indicate that the data of a given node depends on the data of another node [5].

In a CFG the mutation of variables across various basic blocks is not tracked directly. Data flow can be tracked in a CFG, by converting the IR to SSA form.

### 2.2.3. Static Single Assignment Form (SSA)

In SSA, each assignment creates a new, distinct version of a variable, ensuring that every variable instance is immutable. Different definitions of a variable are combined using a $\phi$-function, which is inserted at the beginning of a basic block that has multiple predecessors defining the same variable [1]. The following example illustrates the conversion of a simple loop to SSA:

Table 2.2.: Three-address code to SSA

```
<1>:
   t_1 = a < 4
   if t_1
    goto <2>
   else
    goto <3>
<2>:
   a = a + 1
   goto <1>
<3>: /* ... */
```

```
<1>:
   a_1 = phi<a, a_2>
   t_1 = a_1 < 4
   if t_1
     goto <2>
   else
     goto <3>
<2>:
   a_2 = a_1 + 1
   goto <1>
<3>:
   /* use a_1 for a */
```

In this example, a new version of the variable `a` is introduced in basic block 2. Because block 2 jumps back to block 1, a $\phi$-function is inserted at block 1 to merge the different definitions into `a_1`. Blocks 2 and 3, which are only reachable from block 1, continue to use `a_1` without introducing additional $\phi$-functions. Although it would be valid to place a $\phi$-function at the start of every basic block, this is unnecessary and inefficient.

Several algorithms exist to convert a CFG to SSA [18]. A simple, though non-optimal, approach is to insert a $\phi$-function for every variable at every join point. To minimize the number of $\phi$-functions, dominance relationships between nodes must be computed: a node `a` dominates another node `b` if every path from the entry node to `b` passes through `a`. The dominance frontier of `a` is the set of nodes that are not dominated by `a` but have at least one predecessor domi-

nated by a. When a variable is assigned in a node, a $\phi$-function is inserted at each node in the dominance frontier if the variable may be reached through multiple paths [11, 18].

Pointer assignments require special handling: any variable a pointer may alias must be stored and loaded into a new SSA variable at its next use. Indexed SSA variables are often kept in registers, with store operations signaling when values must be written to memory. Optimizations aim to minimize unnecessary stores by analyzing pointer aliasing, which can also leverage value propagation techniques [1].

If an optimization violates SSA properties, the SSA form must be restored before performing subsequent optimizations that rely on it.

When representing an IR as a CFG in SSA, it is possible to detect constraints and infer information that cannot reliably be determined otherwise. Many optimizations can be applied to a CFG, but this section highlights only the most common.

### 2.2.4. Value Propagation

Value propagation works by tracking assignments and their uses. It can take the form of constant propagation, copy propagation, or value range propagation. These optimizations often take advantage of Static Single Assignment Form (SSA).

**Constant propagation** in SSA is straightforward: each variable use is replaced with the constant assigned to it. While arithmetic optimizations can be handled in a separate pass, constant propagation often precomputes arithmetic expressions when all operands are constants [1].

**Copy propagation** replaces uses of a variable that is simply a copy of another with direct uses of the original. This allows subsequent computations to refer directly to the original variable, which may enable further optimizations such as dead-code elimination if the copied variable is no longer used.

For example, consider the following code:

Table 2.3.: Copy Propagation Example

| | |
|---|---|
| `1: b = a`<br>`2: c = 3 + b` | `1: b = a`<br>`2: c = 3 + a` |

After propagation, c directly uses a instead of b. If b has no other uses, it can be eliminated in a later dead-code elimination pass. This combination of copy propagation and dead-code elimination is sometimes referred to as *single-use forwarding*. Copy propagation also helps remove redundant assignments introduced by other optimization passes.

**Value range propagation** does not change the code directly but annotates variables with the range of values they may hold. These annotations assist further optimizations. For example, if a comparison always succeeds within a known range, the compiler can simplify it. In SSA, the range of a variable may change throughout a procedure, so ranges are tracked at each expression rather than statically for the variable.

- Initially, an uninitialized variable may hold any value.
- Assigning a constant collapses the variable's range to that value.
- Propagation through expressions updates ranges, e.g., $y \in [1, 3]$ and $x = y + 2$ implies $x \in [3, 5]$.
- Branch conditions constrain ranges, e.g., for $a \in [-1, 1]$, inside `if(a>0)`, $a \in (0, 1]$.
- When multiple branches merge, the range of a $\phi$-function is the union of incoming ranges.

Range propagation requires fixed-point iteration. In some cases, convergence may be impractical:

```
1: a_0 = phi<1000000000, a_1>
2: a_1 = a_0 - 1
```

Figure 2.2.: Non-Convergence in Value Range Propagation

Here, `a_0` would converge only after 1,000,000,000 iterations. Practical compilers limit iterations by increasing the step size of range updates or ultimately giving up and annotating the variable as potentially holding any value [1].

Value range propagation enables extensive dead-code elimination by revealing unreachable branches. This includes not only numeric ranges but also the possible types of variables. Type propagation is a form of value range propagation in which the "range" consists of the potential types a variable may hold rather than numeric or constant values. These analyses are particularly useful for array bounds checks in iterators and type checks in dynamically typed languages. After propagation, comparative statements or type checks that depend on constants or known types can be simplified, so subsequent Dead-Code Elimination (DCE) passes require only minimal checks.

### 2.2.5. Dead Code Elimination

Dead-Code Elimination (DCE) is an optimization that removes unnecessary assignments, unreachable branches, and unused functions. Removing unused functions requires interprocedu-

ral analysis.

An assignment whose result is never used can be removed. This includes assignments that are later overwritten, as well as so-called no-op statements such as $x = x$. In Static Single Assignment Form (SSA), every assignment creates a new indexed version of a variable. If that version is never subsequently used, the assignment can be eliminated. When no versions of a variable remain, the variable itself can be removed, reducing memory usage. Note that redundant assignments can only be removed if the variable is not marked as `volatile`.

DCE passes rely on constant propagation and value range propagation to identify dead branches, since these optimizations may convert branch conditions into constants. Dead branches can be removed along with any basic blocks they point to, provided those blocks are otherwise unreachable. $\phi$-functions must also be updated or removed to avoid referencing undefined blocks or variable indices. When a $\phi$-function is removed, the new variable version it created becomes unused and can be eliminated; typically, this removal occurs during a subsequent copy propagation pass rather than in DCE itself [1].

### 2.2.6. Interprocedural Optimizations

The concept of a CFG can be extended to interprocedural analysis by modeling each procedure as a node in a call graph, with edges representing procedure calls. This allows optimizations to reason across procedure boundaries. If a procedure is never called and cannot be reached through indirect calls (e.g., via function pointers), it can be removed. Conversely, small procedures may be inlined at their call sites. If a procedure is not recursive and all call sites are eliminated through inlining, the procedure itself can be removed.

Value propagation can also be applied interprocedurally under certain assumptions. For example, if a procedure is always invoked with the same constant argument, that constant can be propagated into the procedure body, allowing the corresponding parameter to be eliminated. More generally, the value range of a parameter can be approximated as the union of all argument ranges observed at its call sites. This can enable further simplifications, potentially reducing a procedure to a constant result that can be inlined.

However, interprocedural analysis is more expensive than its intraprocedural counterpart. In practice, it requires fixed-point iteration across multiple procedures, which can be computationally expensive.

Interpreters can mitigate this cost through speculative optimization. Instead of requiring global convergence, they generate specialized versions of procedures based on the argument values or ranges observed at runtime. At call time, the interpreter checks whether these as-

sumptions still hold and either executes the specialized version or falls back to a more general implementation.

### 2.2.7. Register Allocation

An interpreter must decide where to store the intermediate values produced during execution. Whether backed by hardware registers or an array of virtual registers, the storage available for live values is finite. Register allocation is the process of assigning variables and intermediate values to these storage locations such that simultaneously live variables do not share one, spilling to memory only when necessary. Optimal register allocation can be framed as a graph colouring problem on a register interference graph, where nodes represent variables and edges connect pairs that are simultaneously live. This problem is NP-complete, making optimal allocation impractical for complex procedures. In practice, interpreters and JIT compilers favor Linear Scan Register Allocation (LSRA) due to its low compilation overhead.

LSRA iterates over the CFG to determine the *live interval* of every variable — the range of the control flow graph from the variable's initial assignment to its last use. After computing all live intervals, the algorithm assigns registers to variables in a single linear pass. When the number of simultaneously live variables exceeds the number of available hardware registers, a variable must be spilled: its value is written to memory and loaded back into a register only when needed. LSRA selects the spill candidate by choosing the variable with the longest remaining live interval, on the basis that this frees the register for the greatest possible duration. However, the algorithm does not perform frequency-based prioritization, meaning that a heavily used variable inside a loop may be spilled in favor of one that is rarely executed. This makes LSRA less suitable for ahead-of-time compilers targeting peak throughput, where more expensive allocation strategies are justified.

Second-Chance Bin Packing (SCBP) addresses one of LSRA's main inefficiencies. In LSRA, a spilled variable that is subsequently used must be loaded into a register, used, and then immediately spilled again if no register remains free. SCBP avoids this by inspecting, at the point of the spill decision, whether a register will become free before the variable's next use. If so, the variable can be scheduled to occupy that register directly, eliminating the redundant load-use-spill sequence.

For interpreters that store intermediate results in an array of virtual registers rather than hardware registers, LSRA serves a different but equally useful purpose: bounding the size of that array. Because the array only needs to be as large as the maximum number of simultaneously live variables, LSRA's live interval analysis determines this maximum precisely. Since the num-

ber of virtual registers is never exceeded by the number of live variables, no spilling occurs and the array size is kept minimal [17, 20].

## 2.3. Memory Management

Unlike early programming languages like C, languages usually provide mechanisms to handle memory management automatically. This section discusses different approaches to memory management and their respective benefits and drawbacks.

### 2.3.1. Reference Counting

A reference-counting garbage collector wraps every heap value in a header struct that contains, besides the payload, an atomic counter of live references.

Before the value is consumed, the emitter inserts an explicit `ref` instruction that increments the counter; after the last use it emits `deref`, which decrements the counter and, if the count reaches zero, immediately invalidates the underlying memory. Function calls consume a reference to a reference counted value, though calling conventions may differ between implementations.

Reference counting itself is agnostic to mutability. The counter only tracks how many pointers exist, not what is stored inside the object. Frameworks that take advantage of reference counting eliminate unintuitive mutations through Copy-On-Write (COW). Many readers may share the same physical buffer, but if anyone tries to mutate it while not holding exclusive ownership over the object ($refcount \neq 1$), a new private copy is created whose reference count is one, while the reference count of the shared object is decremented.

This scheme is simple and deterministic, but it cannot cope with cyclic structures: when two objects point to each other their counters never drop to zero, causing a memory leak.

Because most general-purpose languages allow arbitrary references (and therefore cycles), pure reference counting is inadequate. Reference counting still finds application in some libraries and domain specific languages that operate within these constraints. An illustration of these concepts can be found here A.1.

### 2.3.2. Garbage Collection

Garbage collection is the process of automatically reclaiming memory that is no longer reachable by a program. The heap can be modeled as a graph, where objects are nodes and references between them are edges. To determine which objects are alive, the GC traverses

this graph starting from a set of roots — pointers held in the stack, global variables, and registers. Any node reachable from these roots is marked as live. Once the traversal is complete, the sweep phase invalidates the underlying memory of every unmarked node, reclaiming it for future allocations. To keep the object graph accurate, the runtime must intercept every operation that changes a reference. When a pointer is written, an object field updated, or a variable reassigned, the runtime needs to record that edge change in the graph. This is done via write barriers: small pieces of instrumented code injected around pointer writes that notify the GC of the mutation. Without them, the GC could traverse a stale graph and either collect live objects (catastrophic) or miss garbage. This gets especially complex in concurrent GC. Should an exception cause the stack to unwind, the GC runtime needs to insert instructions into the relevant landing pads to keep the graph up to date. Most modern runtimes build on this with generational GC, motivated by the generational hypothesis: most objects die young. Rather than scanning the entire heap every collection, objects are assigned to generations based on how long they've survived. A minor GC runs frequently and applies mark-and-sweep only to the youngest generation, where most garbage is expected to be found. A major GC runs less often and sweeps across all generations. Any object that survives a collection is promoted to the next generation, where it will be checked less frequently — keeping the common case fast while still eventually collecting long-lived objects. Garbage collection is widely used in languages that don't require manual memory management, however garbage collection introduces substantial overhead and requires a GC-runtime[2] [26].

### 2.3.3. Resource Acquisition Is Initialization (RAII)

Memory safety requires a disciplined answer to two questions: who is responsible for releasing a resource, and for how long is a reference to it valid? Three interlocking concepts address these questions: ownership, borrowing, and lifetimes.

Every resource has exactly one owner at any point in time. That owner is solely responsible for releasing the object by calling its destructor. Under RAII, a resource is acquired when an object is initialized and released when that object is destroyed. Ownership can be *transferred* to another binding, after which the source is no longer valid and the destination becomes the sole responsible party. Languages such as Rust enforce this statically; C++ provides the same mechanism through move constructors but relies on programmer discipline or static analysis tools to prevent use after move.

*Borrowing* allows a resource to be temporarily accessed without transferring ownership. A

[2]This is especially a concern in embedded environments. For user-space code libraries exist to handle this added complexity

borrow creates a reference to the resource and passes it to a callee; the original owner retains responsibility for eventual cleanup and reclaims full control when the borrow ends. To prevent data races and dangling references, a type system can restrict borrows: typically either many simultaneous immutable references or exactly one mutable reference is permitted at a time, but not both.

A *lifetime* is the static region of the program during which a reference is guaranteed to be valid. A sound type system must ensure that no reference outlives the value it points to; a reference that outlives its referent is a dangling pointer. Accessing it is undefined behavior. In practice, lifetimes are either inferred by the compiler or annotated explicitly by the programmer to express constraints between the lifetime of a reference and the lifetime of the value it was derived from [9, 22].

Under normal control flow, the compiler automatically inserts destructor calls at every exit point of a scope, whether that exit is a normal return or an early return. Non-local exits – such as exceptions – bypass the normal function epilogue entirely, meaning the compiler cannot statically insert cleanup at the exit site. Instead, the runtime must walk the call stack and invoke destructors for every live owned value between the exit site and the handler. This mechanism is discussed in the following section. Importantly, RAII only guarantees cleanup for values managed by the host language's type system. An interpreter that manages guest-language values explicitly – for example, through a manually maintained heap – is responsible for replicating this cleanup behavior when a guest exception propagates, since those values fall outside the host's ownership tracking.

## 2.4. Non-local Control Flow and Exception Handling

As established in the preceding section, RAII guarantees that owned values are destroyed when they go out of scope, but non-local exits bypass the normal epilogue and require a separate mechanism to uphold this guarantee. When control leaves a scope through an exception, panic, or long jump, every owned value between the exit site and the matching handler must still be destroyed exactly once. This is achieved through *stack unwinding*: the unwinder walks the call stack from the exit site toward the handler, invoking the destructor of each live owned value along the way. For an interpreter that supports guest-language exceptions, this introduces two distinct unwinding concerns. The first is unwinding through the interpreter's own host-language frames – in a Rust implementation, Rust's ownership system and the underlying unwinding infrastructure ensure that the interpreter's internal allocations are cleaned up correctly. The second is unwinding through the guest language's call frames: the interpreter is responsible for

walking its own representation of the guest call stack and releasing any guest-language values that go out of scope as the exception propagates, since these are managed explicitly and lie outside the host type system's purview.

### 2.4.1. Two-phase unwinding

A naive implementation would destroy objects immediately when an exception is thrown. This is wasteful if no handler exists, since the program will terminate anyway. The Itanium C++ Application Binary Interface (ABI) [8], followed by both GCC and Clang, avoids this by splitting unwinding into two phases. On System V platforms, Rust uses the same Itanium ABI unwinding infrastructure, meaning that panics and exceptions unwind safely through mixed Rust and C++ frames. This is directly relevant for an interpreter written in Rust that calls into native libraries or exposes a native API: unwinding will cross these boundaries correctly provided that landing pads are maintained on both sides [12].

In the *search phase*, the call stack is walked without modifying any state or running any destructors until a frame containing a handler for the exception is found. If the search reaches the top of the stack without finding one, the program terminates immediately. If a matching handler is located, the *cleanup phase* walks the stack a second time, invoking destructors and running cleanup code for every object that goes out of scope, until it reaches the handler frame and transfers control to it [8, 12].

### 2.4.2. Landing pads

A Landing Pad is a small block of compiler-generated code that serves as the entry point for the unwinder when it reaches a frame that requires attention – either because the frame contains a matching handler, or because it has live owned objects that must be destroyed. When the cleanup phase reaches such a frame, the unwinder transfers control to the landing pad. The landing pad runs the necessary destructors, then either dispatches to the appropriate catch block if the exception type matches, or returns control to the unwinder to continue up the stack if it does not. A landing pad is therefore required at any call site that could throw and that has live owned objects in the enclosing scope. In practice, most calls that allocate memory or acquire resources fall into this category [8, 12].

If the interpreter is extended with a Just In Time Compilation (JIT) backend, generated machine code must also register unwind information so that the unwinder can traverse JIT-compiled frames correctly. On Linux this is achieved by registering dynamic frame information at runtime. Without it, an exception propagating through a JIT frame will either terminate the

process or corrupt the stack, since the unwinder has no record of that frame's layout or owned values [13].

## 2.5. Interpreter Models

An interpreter executes a program without first compiling it to native machine code. Instead, it processes the program representation directly, either by walking a syntax tree or by dispatching over a sequence of lower-level instructions. The choice of model has practical consequences for implementation complexity, execution speed, and the ease with which language semantics can be expressed.

### 2.5.1. Tree-Walking Interpreters

The simplest interpreter model operates directly on the AST produced by the parser. Each node in the tree corresponds to a syntactic construct, such as a binary expression, a loop, or a function call. The interpreter evaluates the program by recursively visiting each node and performing the corresponding operation. A binary addition node, for example, would recursively evaluate its left and right children, then return their sum.

Tree-walking interpreters are straightforward to implement because the structure of the interpreter mirrors the structure of the language grammar. There is no separate compilation step: the parser's output is also the execution representation. The main drawback is speed: recursive dispatch over heap-allocated tree nodes incurs significant overhead compared to a flat IR, which is more amenable to optimization [2].

### 2.5.2. Bytecode Virtual Machines

A bytecode Virtual Machine (VM) introduces an IR between the source program and execution. The AST is compiled to a flat sequence of instructions — bytecode — which the interpreter then executes in a dispatch loop. This avoids the overhead of recursive tree traversal and produces a more compact, cache-friendly representation.

**Representation of Intermediate Values** Intermediate values in a virtual machine are the transient data produced and consumed by each instruction. How these values are stored and addressed is the principal design axis that separates stack-based from register-based bytecode VMs.

In a *stack machine*, every operation implicitly works on the topmost elements of a last-in, first-out operand stack. The bytecode itself never names the storage cells; it only states what to do (e.g. `ADD`, `LOAD_CONST` 7). At execution time the interpreter maintains a stack pointer. A binary addition compiles to a single-byte opcode and is executed as

$$\text{temp} \leftarrow stack[SP-1] + stack[SP]; \quad SP \leftarrow SP - 1; \quad stack[SP] \leftarrow \text{temp}.$$

Because operands are always at known offsets relative to the stack pointer, instructions are extremely compact — typically a one-byte opcode plus an optional embedded literal — and the lowering pass from AST to bytecode is straightforward to implement. The consequence is that the operand stack becomes a frequently accessed data structure: every push and pop touches memory, and expression trees must be serialised in postfix order, often generating more instructions than a register-based encoding of the same computation would require.

In a *register machine*, instructions have explicit operand indices instead of an implicit stack. The same addition is expressed as `ADD r3, r1, r2`, which maps directly to

$$R[3] \leftarrow R[1] + R[2],$$

with no change to a global stack pointer. Because results can be held in named registers until they are consumed, an expression tree can often be evaluated with one instruction per operator, reducing the total instruction count relative to a stack encoding. The trade-off is that instructions are larger — typically 8–32 bits per operand field — and the compiler must perform liveness analysis and register allocation to assign virtual registers efficiently.

Regardless of which model is used, bytecode instructions should each serve a single, type-specific purpose. An `iadd` instruction adds two integers; if an operand holds a floating-point value, a separate conversion instruction must precede it, or a dedicated mixed-type instruction must be used. This regularity simplifies the interpreter dispatch loop and makes type errors detectable at the instruction level [1].

# 3. Implementation

## 3.1. Parser and Passes over the Concrete Syntax Tree (CST)

The SetlX reference implementation uses the ANTLR4 parser generator to generate an AST based on the input string. As ANTLR does not properly support Rust as a code generation target, this project uses LALRPOP instead. While ANTLR supports Adaptive LL(*) (ALL) grammars, LALRPOP supports LR(1)[1] grammars. While the two grammars describe the same language, the differing parsing strategies imposed several structural changes. This section discusses the most significant of these, with a focus on reduce-reduce conflicts arising from lambda parameters and assignment statements.

### 3.1.1. Reduce-Reduce Conflicts

A reduce-reduce conflict arises when two or more grammar rules are eligible for reduction given the same parser state and lookahead token. This typically occurs when two constructs share a common prefix that is only disambiguated by context beyond the one-token lookahead window. In the cases described below, the conflict is resolved by introducing shared intermediate rules that force the parser to commit to a single unambiguous reduction. Any shared rule must be a superset of the conflicting rules that it replaces. For both reduce-reduce conflicts described below, the shared rule produces edge cases, where the grammar accepts inputs that don't describe a valid SetlX program. These unchecked constraints must be validated elsewhere.

#### Lambda Parameters

In the ANTLR4 grammar, lambda expressions are introduced via the `lambdaProcedure` rule, which allows the parameter list to be either a single identifier or a bracketed list of identifiers:

[1] "Despite its name, LALRPOP in fact uses LR(1) by default (though you can opt for LALR(1)), [...]" [14]

Table 3.1.: Lambda parameter rules: ANTLR4 EBNF vs. LALRPOP EBNF

```
LambdaParams
  : variable
  | '[' (variable
        (',' variable)*)?
    ']'
  ;

Lambda
  : LambdaParams
    ('|->' | '|=>') Expr
  ;
```

```
LambdaParams
  : Ident
  | BrackCollection
  ;

LambdaIsClosure
    : '|=>'
    | '|->'
    ;

Lambda
  : LambdaParams
    LambdaIsClosure Expr
  ;

BrackCollection
    : '[' Expr ',' ExtendedCollection ']'
    | '[' Expr '..' Expr? ']'
    | '[' '..' Expr ']'
    | '[' Expr ':' IteratorChain PipeExpr? '
        ]'
    | '[' Expr PipeExpr ']'
    | '[' Expr? ']'
    ;
```

**Minimal example isolating the conflict**

```
stmt : lambda | expr;
expr
    : IDENT
    | '[' expr (',' expr)* ']'
    ;
lambdaParams
    : IDENT
    | '[' IDENT (',' IDENT)* ']'
    ;
lambda: lambdaParams '|->' expr ;
```

```
stmt: lambda | expr;
brack
    : '[' expr (',' expr)* ']'
    ;
expr: IDENT | brack ;
lambdaParams: IDENT | brack ;
lambda: lambdaParams '|->' expr ;
```

The conflict arises because a bracketed expression such as `[x,y]` is simultaneously a valid `BrackCollection` and a valid `LambdaParams`. The disambiguating token — `|->` or `|=>` — only appears after the closing bracket, which may be preceded by an arbitrarily long list of elements. In this project, the conflict is resolved by reusing `BrackCollection` as the bracketed lambda parameter rule, accepting that it admits more than valid syntax. The program must subsequently verify manually that the `BrackCollection` only contains a list of variable expressions.

### Assignment Statements

A more structurally significant conflict arises from assignments. The reference grammar expresses the left-hand side via the `assignable` rule, which permits not only bare identifiers and member accesses but also destructuring list patterns such as `[a,b,c]:=[1,2,3]`:

Table 3.2.: Assignment rules: ANTLR4 EBNF vs. LALRPOP EBNF

```
assignable
  : assignableVariable
    ('.' variable
    | '[' exprList ']')*
  | '[' assignmentList ']'
  | '_'
  ;
assignmentDirectContent
  : assignable ':='
    ( assignmentDirectContent
    | exprContent)
  ;
```

```
AssignDecl
  : Accessible ':=' AssignSource
  ;
AssignSource
  : AssignDecl
  | Expr
  ;
```

**Minimal example isolating the conflict**

```
stmt: assignDecl | expr ;
assignDecl: assignable ':=' expr ;
assignable
    : IDENT
    | '[' IDENT (',' IDENT)* ']'
    ;
expr
    : IDENT
    | NUM
    | '[' expr (',' expr)* ']'
    ;
```

```
stmt        : assignDecl | expr ;
assignDecl : assignable ':=' expr ;
assignable
    :IDENT
    | '[' expr (',' expr)* ']'
    ;
expr
    : IDENT
    | NUM
    | '[' expr (',' expr)* ']'
    ;
```

The conflict arises because the `assignable` rule contains a dedicated list pattern `[a,b,c, ...]` whose closing bracket may be preceded by an arbitrarily long sequence of tokens. It is only possible to determine whether the bracketed construct is an assignment target or a plain expression when the `:=` token is eventually read.

When an LR(1) parser reads `[` followed by `IDENT` it cannot determine whether to pursue the `assignable` path or the `expr` path: both are consistent with the tokens seen so far. The disambiguating `:=` only appears after the closing `]`, which may be preceded by arbitrarily many comma-separated identifiers, placing it beyond any fixed lookahead bound.

The conflict is resolved by parsing the assignment target as `Accessible` — the most specific expression rule capable of describing all valid assignment targets — thereby eliminating the dedicated list pattern and with it the unbounded lookahead requirement. The auxiliary rule `A ssignSource` anchors the right-hand side to either another `AssignDecl`, supporting chaining such as `a:=b:=4`, or a plain `Expr`. Invalid left-hand side forms such as `1:=4` or `a():=3` must be validated manually. [2]

[2] In this implementation, parentheses that define precedence influence the structure of the CST at parse-time, but are not stored as explicit CST nodes. Accordingly, the assignment `(a):=1;` is accepted by the interpreter and handled just like the assignment `a:=1;`.

### 3.1.2. Further Constraints Imposed by LALRPOP

Beyond the specific conflicts described above, some quirks of LALRPOP influenced the design of the grammar more broadly.

**Absence of generic rules.** ANTLR4 allows parameterised rules and rule reuse through inheritance and lexer modes. LALRPOP has no equivalent mechanism. Rules that share structure but differ in their delimiters — most notably `BrackCollection` (`[...]`) and `CurlyCollection` (`{...}`) — must be written out in full as separate, duplicated rules.

**Optional rules.** In LALRPOP, optional rules are expressed using the `?` postfix operator on a rule reference, which requires the rule to return a concrete type `T`, producing `Option<T>` at the call site. This means that any rule used in an optional context must have a fully resolved, non-generic return type. Where the ANTLR4 grammar can freely make any rule optional inline, LALRPOP often requires the introduction of an auxiliary rule solely to provide a concrete return type that can be wrapped in `Option<T>`. Examples of this pattern in the grammar include `ClassStaticDecl`, `CheckAfterBacktrack`, and `MatchRegexBranchAs`, each of which exists primarily to satisfy this type requirement rather than to express meaningful grammatical structure.

### 3.1.3. CST Passes

As discussed above, this `LR(1)` grammar accepts invalid inputs. Furthermore, compound strings such as `"hello$1+1$"` have not been parsed yet[3] – which also extends to C-style escape sequences[4]. This project handles these cases by walking the CST, transforming it or its context when necessary. Once the structure has been transformed, this document refers to the result as an Abstract Syntax Tree (AST).

[3] In SetlX terms take precedence over strings for the plus operation. However, inline expressions need to be serialized and concatenated with the string. This problem is resolved by introducing the CST node `CSTExpressionKind::Serialize` that is applied to the expressions in compound strings.

[4] Unlike other programming languages SetlX allows unrecognized escape sequences, by leaving the sequence as is.

Table 3.3.: CST Passes

| Pass | Description |
|---|---|
| pass string | Parse compound strings containing inline expressions into concatenation expressions, and process escape sequences. |
| pass check | Verify the validity of the CST. Further details are discussed below. |
| pass noop | Detect and remove unreachable statements, emitting a warning for each one. |

`Pass Check` ensures correct use of `break` and `continue` within loop bodies, that match case expressions consist only of matchable constructs, the structural correctness of assignment targets as well as accessible body elements, and that lambda parameter lists contain only variable expressions. [5]

Apart from generating errors, these passes can also generate non-fatal warnings. Error reporting diverges from the reference implementation. Instead of printing a minimal backtrace, the crate `ariadne` [3] is used to render error diagnostics with source annotations. Accordingly, SetlX programs that catch dynamic parse errors will behave differently in this interpreter.

## 3.2. Internal Interpretation Concepts

The interpreter described in this project executes SetlX by converting the AST to an IR that is run as a bytecode VM. This approach was chosen out of personal interest in interpreter optimization and because it leaves the door open to future optimization passes and JIT compilation that a tree-walking interpreter would not support. The priority was to establish a sound IR framework that is semantically correct and can be improved incrementally, rather than to match the reference implementation's throughput from the outset.

The IR lowering stage supports all native SetlX features except `scan` default blocks.

### 3.2.1. Control Flow

The IR is represented as a set of procedures, basic blocks, and three-address code statements. The interpreter terminates when the main procedure returns. Every procedure has exactly one start block and one return block — having a single return block ensures that all active contexts, such as try contexts, are properly invalidated before the procedure exits.

[5] The definition of the CST can be found in `src/ast.rs`. The passes are implemented in `src/cst/passes/`.

| Statement | Description |
|---|---|
| Branch | Evaluate a boolean condition. If the condition is true continue execution at the success block. Else continue execution at the failure block. |
| Goto | Continue execution at the block this statement points to. |
| Return | Exit the current procedure. Assign the value of the return statement to the target of the corresponding call statement. |
| Try | Begin a new try context. Continue execution at the attempt block. If an error is thrown within the try context, the exception is caught and execution continues at the catch block. |
| TryEnd | Terminate the most recent try context and jump to a basic block outside. |
| Unreachable | The statement cannot be reached. This statement is an indicator that the prior statement throws an exception or terminates the process. |

Table 3.4.: Control Flow Terminal Statements

Procedures, blocks, and statements exist at different layers of abstraction: a statement may reference a block or a procedure, but cannot define either. Nested procedures exist on the same layer as all other procedures. Basic blocks within a procedure are stored in a directed graph, where each basic block is a node. Each block $A$ that may transfer execution to a block $B$ is represented as an edge $A \rightarrow B$. Procedures are likewise stored in a directed graph, with an edge from caller to callee for each procedure reference.

Procedures are reference counted. Unlike other heap values, copying or invalidating a procedure reference does not affect the underlying procedure body — both operations merely adjust the reference count, and the body is shared among all references until the count reaches zero.[6] As SetlX does not support arbitrary references[7], no cyclic dependencies between procedures can occur.

An example illustrating the basic structure of a procedure can be found in appendix A.2. IR dumps can be generated by running the interpreter with the flag `--dump-ir-lower`.

Higher-order control flow statements such as loops or conditional statements must be trans-

[6] Procedure calls are resolved dynamically and do not create static edges in the procedure graph, so recursive calls do not produce reference cycles.

[7] In SetlX class objects are not passed by reference. Instead, an independent copy of the object is created, should the need arise.

formed to this form as well. A simple while loop consists of two sections: the condition block that is evaluated and the body block that executes if the condition succeeds. The body block terminates with a `goto` back to the condition block. If the condition fails, execution continues in the follow block. A `break` statement is transformed to a `goto` to the follow block, and a `continue` statement is transformed to a `goto` to the condition block.

The following example shows how a while loop is transformed by the interpreter:

Table 3.5.: Lowering While Statement IR Example

| | |
|---|---|
| `while (true) {`<br>`  break;`<br>`}` | `<bb2>: // condition block`<br>`  t1 := true;`<br>`  if t1`<br>`  goto <bb4>`<br>`  else`<br>`  goto <bb3>`<br><br>`<bb4>: // while block`<br>`  goto <bb3> // break`<br><br>`<bb3>: // follow block`<br>`  goto <bb5>` |

### 3.2.2. Heap Memory Management

Values that require dynamically sized storage – strings, lists, sets, objects, arbitrary-precision integers, and procedures with captured environments – are allocated on the heap. Virtual registers contain references to these heap objects. As SetlX does not define language features that would allow cyclic references, memory is managed manually at the IR layer[8]. The interpreter tracks ownership when transforming the CST to IR to avoid memory leaks or double frees. No runtime safety checks guard these operations: the correctness of allocation and deallocation depends entirely on the IR lowering stage emitting valid ownership instructions.

**Ownership invariant.** At every point during execution, each live heap object is owned by exactly one of the following: a virtual register, a stack variable's value slot, or a field within another heap object (e.g., an element of a list or a member of an object). All other references

[8]Rust's native reference counting interface is not used here. Since virtual registers may hold references beyond their last semantic use, `Rc` would delay deallocation in ways that are difficult to reason about at the IR level. Explicit ownership instructions give the code generator precise control over when heap objects are freed, independent of register liveness as seen by the host language. Furthermore, the interpreter would still have to maintain COW semantics during code-gen.

to the object are borrowed – immutably borrowed references may read the value but must not invalidate it. Views such as slices and iterators are immutable borrows of this kind. Slices and iterators are explained in section 3.3.1. This invariant is maintained by the IR code generator and is not enforced at runtime.

**Copying and invalidation.** A borrowed reference can be promoted to an owned reference by calling `copy`, which allocates a fresh heap object with the same contents. If the copied object is a container – a list, set, or object – the copy is deep: every heap-allocated element is itself copied recursively, and the new container takes ownership of the copies. Invalidation deallocates the heap object and recursively invalidates all owned children. After a virtual register's owned value has been invalidated or its ownership transferred, the register must not be read until it is reassigned. This constraint is a code-generation guarantee; violating it produces use-after-free or double-free bugs.

**Ownership transfer.** When a value is inserted into a container – for example, via a builtin procedure such as `list_push` or `set_insert` – the container takes ownership. The caller must not use or invalidate the transferred value afterward. The same principle applies when a value is written into a stack variable's slot via pointer dereference assignment (`*t:=v`): the stack slot takes ownership, and the source register must not be reused.

**Mutable borrowing.** Reassigning a variable that already owns a heap value requires invalidating the old value before writing the new one. This is modeled as a mutable borrow: the old value is read through the pointer, invalidated, and then the new value is written. This sequence must be atomic in the sense that no operation between the invalidation and the write can raise an exception; otherwise, subsequent code or cleanup logic could observe the slot in an undefined state. Mutable borrowing is only safe when no other active borrows of the old value exist. The IR code generator is responsible for ensuring this – no runtime check verifies it. An example illustrating this pattern is provided in Appendix A.3.

**Immediate heap.** Each procedure call and each try context maintains an immediate heap: a set of references to heap objects that are owned by virtual registers but have not yet been transferred to a persistent owner (a stack variable or container). When a heap object is first allocated into a virtual register, it is added to the current immediate heap. When ownership is transferred – to a stack slot, a container, or a callee – the object is removed from the immediate heap. When a try context or procedure exits, its immediate heap is dropped, which

invalidates all remaining entries. This serves as a safety net: if an exception causes an early exit, any temporary allocations that were never transferred to a persistent owner are cleaned up automatically, preventing leaks.[9]

To illustrate the aforementioned concepts, consider the following example:

```
<bb1>:
    t_1 := list_new();          // allocate heap 0 owned by t_1
    t_2 := t_1;                 // t_2 references heap 0 t_2 is borrowed
    t_2 := copy(t_2);           // allocate heap 1 owned by t_2
    t_3 := t_1;                 // transfer ownership to t_3
                                  // t_1 must not be used or borrowed until
                                       reinitialization
    _ := invalidate(t_3);       // invalidate heap 0
    t_1 := "hello world";       // allocate heap 0 owned by t_1
    _ := list_push(t_2, t_1); // transfer ownership to list owned by t_2
                                  // t_1 must not be used or borrowed until
                                       reinitialization
    return om;                  /* memory owned by t_2 isn't persistent
                                    -> invalidates heap 1
                                     -> invalidates heap 0
                                 */
```

Figure 3.1.: Heap Ownership IR Example

**Persist and immediate markers.** These builtins manually remove or add an object from the immediate heap. They are needed in two situations. First, when a value must survive the teardown of its enclosing try context – for example, a return value computed inside a `try` block must be marked persistent before the try context exits, then re-marked as immediate in the caller's context once the callee returns. Second, when a builtin procedure returns a borrowed reference, the caller must mark it as persistent to remove it from the immediate heap, since the value is still owned by its original container and would otherwise be invalidated twice on cleanup.

**Procedures as heap objects.** Procedures are heap-allocated but reference-counted. Copying or invalidating a procedure reference adjusts the reference count rather than duplicating or deallocating the procedure body. Since SetlX does not support first-class references that could create cycles between procedures, no cyclic dependencies can arise, and reference counting

[9] Static compilers like LLVM handle this by invoking landing pads before rethrowing exceptions to invalidate non-persistent memory. Tracking immediate heap memory reduces the bookkeeping required when lowering the IR.

is sufficient.[10]

### 3.2.3. Stack

The stack is a list of stack entries. A stack entry can be a frame boundary, variable, or alias. The stack is the primary mechanism for managing the lifetimes and accessibility of variables.

Frame boundaries segment the stack into regions called stack frames. Only variables and aliases after the last frame boundary, or entries marked as accessible across frame boundaries, are accessible. `stack_frame_add()` pushes a new boundary, establishing a new frame, while `stack_frame_pop()` tears it down, removing all entries after the boundary and invalidating any heap references they held.

Variables are named slots holding an owned value. `stack_add()` pushes a new variable onto the stack and returns a pointer to its value slot. If an accessible variable already exists on the stack, the new entry inherits a copy of its value. This ensures that cross-frame entries such as predefined procedures remain accessible across frame boundaries. Memory referenced by the value slot is owned by the variable. `stack_pop` removes the currently accessible variable of that name from the stack. Should the slot contain a heap reference, this heap reference is invalidated. `stack_get_or_new` retrieves a pointer to a named variable if one is accessible in the current frame, and otherwise falls back to the same logic as `stack_add()`. `stack_in_scope` indicates whether a named variable is accessible in the current frame.

Aliases are named pointers to an existing value slot, owned by another variable or heap object. `stack_alias()` registers an alias with an optional `crosses_frames` flag. When set, the alias remains accessible across frame boundaries. Invalidating a stack alias does not invalidate the memory referenced by it.

Stack images are snapshots of reachable stack entries. `stack_frame_copy()` produces a non-destructive copy of the current frame's entries, whereas `stack_frame_save()` also tears down the current stack frame, transferring ownership of the entries onto the image. A stack image can be embedded into a procedure at construction time and later retrieved with `procedure_stack_get()`. `procedure_stack_get()` creates a copy of the stack frame if one is embedded in the procedure. Closures in SetlX carry snapshots of the stack at the time of definition instead of capturing the environment at call-time. `stack_frame_restore()` pushes copies of all entries from a stack image onto the current stack frame. After `stack_frame_restore()`, any previously obtained stack pointers are considered outdated. The prior pointers remain valid and still point to their original slots, but those slots may now be inaccessible. Any

[10] Procedure calls are resolved dynamically and do not create static edges in the procedure graph, so recursive calls do not produce reference cycles.

code holding such pointers should instead look up or re-derive references to the corresponding restored entries, which represent the authoritative current state of those variables.

Predefined procedures are defined ahead of the first stack frame and marked as cross-frame. This makes them accessible from any stack frame, unless a stack entry exists in a subsequent frame which overwrites them.

A practical example of the behavior of closures is provided in appendix A.4

### 3.2.4. Calling Conventions

A procedure is called with a single parameter, which is either a list of pointers or undefined if no arguments are passed. Within a procedure, this parameter list is accessible through the builtin variable `BuiltinVar::Params`. The values referenced by the pointers in the parameter list may be mutated by the callee. The caller must invalidate any parameter values it owns after the call completes. Parameters must not be marked as immediate by the caller, as marking them as such may cause the prior value of a mutated parameter to be invalidated twice.

As parameter values may be overwritten by the callee, the caller wraps each procedure call in an IR-level try/catch block. The catch block acts as a landing pad, performing the same invalidation of owned parameters that would otherwise occur on the normal return path.

If the procedure is a SetlX closure (see section 3.2.3), the first parameter holds a copy of the embedded stack image, ownership of which is transferred to the callee.

### 3.2.5. Exception Handling

Exceptions are implemented using Rust's panic mechanism[11]. When an exception is raised, a global exception state is set — consisting of an exception kind and an exception value — before a panic is triggered. There are three exception kinds: `Lng` for internal interpreter errors, `Usr` for language-level errors caused by invalid operations or user-thrown exceptions, and `Back track` for the backtracking control flow mechanism used by the `check` statement.

A `try` statement in the IR pairs an attempt block with a catch block. At the interpreter level, the attempt block is executed inside a `catch_unwind` boundary. If execution completes normally, control passes through a `TryEnd` statement to the appropriate successor block — whether that is the normal follow block, a return block, a break block, or a continue block. If a panic occurs, control transfers to the catch block. Each `try` context also establishes a fresh immediate heap, so any temporary values allocated during the attempt are scoped to that region. Prior to returning from a procedure, every try context must be terminated.

[11] The interpreter assumes that Rust isn't compiled with `panic=abort`.

Catch blocks check the current exception kind against the kind they handle. If the kind matches, the exception value is pushed onto the stack under the catch variable name, the global exception state is reset, and the catch body executes. On exit from the catch body, the exception variable is popped from the stack. If the kind does not match, the exception is rethrown by triggering a new panic with the existing global exception state, allowing it to propagate outward to the next enclosing `catch_unwind` boundary.

A normal SetlX `catch` statement catches only `Usr` exceptions, while `check` catches only `Backtrack` exceptions. Internal interpreter errors can be caught with `catchLng`. At the top level, any uncaught panic is intercepted and its exception state is inspected to produce a human-readable error message: user errors are serialized like any other SetlX value, language errors print the serialized exception value, and internal errors print the panic payload directly.

### 3.2.6. Classes

#### Class Definition

A class definition compiles into two procedures: a static procedure and a constructor procedure. The static procedure runs the class's static block, saves the resulting stack frame as an image, and returns it. The entries in this stack frame become the static members of the class, which are direct assignment targets accessible from outside the class and copied into each new object upon construction.

The constructor procedure pushes the static block's stack frame, runs the class body — defining methods and fields as named variables — saves that frame as an image, and constructs a new object from it via `object_new`. An object is a flat map of named members to their associated values. Prior to construction, a `getClass` member is added to the object, containing a copy of the class's `get_proc`. When called, `get_proc` returns a copy of the class, so for an object `o` derived from class `c`, calling `o.getClass()` returns a copy of `c`.

#### Class Store

When a class definition is encountered at runtime, `class_add` registers the class in the class store, which is a map of class names to class store entries. Each entry holds a `get_proc` stub and a reference to the class object, which in turn holds the constructor procedure and the static variables. The class store takes priority over the stack during variable lookup: once a class has been registered under a name, no stack variable may be added under the same name. Attempting to do so causes an exception, ensuring class names remain unambiguous

throughout execution.

#### Method Calls

When a method is called on an object, the object's members and `this` – which references the object itself – are made accessible to the method by reference. A new stack frame is established and the object is aliased as `this`, with each of its members aliased by name, all marked cross-frame. The method and any procedures it calls can therefore access `this` and all object members as ordinary variables, with no explicit qualification. When the call returns, the frame is torn down, removing all aliases at once.

#### Operator Overloading and Serialization

Operator overloading is convention-based: an object supports an operator by having a procedure member with the corresponding name. When an operator is applied to an object, the runtime looks up the matching member and calls it as a method with the right-hand side as its argument. If no matching member exists, the operation either falls back to a default behavior or throws an exception. Addition is a special case: it has two fallback paths. If the object defines a `sum` member procedure, that is used as the overload. Otherwise, if the object defines a member `f_str` and the right-hand side is a string, the object is serialized via `f_str` and concatenated with the right-hand side string.

Serialization follows the same convention.[12] If the object has a `f_str` member procedure, it is called as a method with no arguments and its result is used as the string representation. Otherwise the object falls back to the default representation `object<name:=value;...>`.

## 3.3. Interpretation

The IR described in the previous section is interpreted directly. Unlike a typical bytecode interpreter, instructions are not type-specialized — the same instruction may perform different operations depending on the runtime types of its operands. This directly mirrors the dynamic semantics of SetlX at the cost of more complex instruction implementations. Each procedure maintains its own array of virtual registers and an immediate heap, both of which are allocated on the Rust stack for the duration of the call. The following subsections describe how SetlX values are represented at runtime and how the predefined procedures are initialized.

[12] Serialization refers to the practice of converting a SetlX value to a string.

Instructions are dispatched via a Rust `match` statement over the instruction variant. Arithmetic and comparison operations resolve their operands through nested `match` statements: the outer arm matches the type of the left-hand operand, and the inner arm matches the type of the right-hand operand. Each combination of operand types either applies the appropriate operation — for example, integer addition when both operands are integers, or a widening conversion followed by floating-point addition when one operand is a double — or produces a runtime type error if the combination is not defined for the given operator. This mirrors the dynamic semantics of SetlX directly, at the cost of more branching per instruction than a type-specialized dispatch would require.

### 3.3.1. Value Representation

At runtime, SetlX values are represented as `InterpVal`, a tagged union that covers both stack-allocated and heap-allocated variants. Stack-allocated variants include booleans, doubles, pointers, and `Undefined` — the runtime representation of SetlX's `om`. Heap-allocated values are represented as `InterpVal::Ref`, which holds a pointer to an `InterpObj`. The heap object variants cover the dynamically-sized types of the language: arbitrary-precision integers, strings, lists, sets, objects, classes, procedures, matrices, vectors, and regular expressions. In the IR values are represented as an enum `IRValue`. The interpreter converts an `IRValue` to an `InterpVal` at runtime should the need arise. If an operation operates on internal types – such as strings, numbers, or floats – the interpreter may convert `IRValue` to that target type directly instead.

Procedures are heap-allocated and may carry additional data: a procedure lowered from a SetlX closure carries an embedded stack image, and AST metadata. The `InterpVal::Procedure` variant is an implementation detail used internally and should be considered equivalent to `InterpVal::Ref(InterpObj::Procedure)` for the purposes of this discussion.

Two additional variants, `InterpVal::Slice` and `InterpVal::Iter`, exist as optimizations for iteration. Rather than copying a string or list when iterating over it, the interpreter produces a slice or iterator view over the original allocation. These variants have unique lifetime requirements: they hold raw references into heap memory and must not outlive the object they reference. A `Char` variant similarly avoids allocating a heap string for each character produced during string iteration, existing solely as an intermediate value within the iteration context.

### 3.3.2. Predefined and Builtin Procedures

Predefined procedures are the named procedures accessible to every SetlX program without explicit declaration, such as `print`, `abs`, or `load`. At startup, before the first stack frame is pushed, each predefined procedure is constructed as an IR procedure stub and added to the stack as a cross-frame variable. This ensures they remain accessible from any stack frame throughout execution and are inherited by closures via stack snapshots. Predefined procedures participate in the same calling convention as user-defined procedures and can therefore be passed as values, stored in variables, and captured by closures.

Internally, predefined procedure stubs delegate to `BuiltinProc` calls for their implementation. `BuiltinProc` variants are lower-level operations invoked directly by the generated IR — such as `stack_add`, `object_new`, or `invalidate` — and are not accessible as named variables in SetlX programs.

Table B.1 lists all predefined procedures and their support status in this implementation.

# 4. Conclusion

## 4.1. Summary and Reflection

This project has been a substantial undertaking. While I initially anticipated SetlX to be a small language, terms, classes, and matching are rich features that introduce considerable complexity.

The main aim of this project as a student paper has been to implement a minimum viable product supporting the main features of the language. Though some predefined procedures are not yet implemented, all SetlX programs written for the Logic lecture of Karl Stroetmann are interpreted correctly, though the output of two of the files contain minor visual artifacts. The SetlX example code is more of a mixed bag, because of unimplemented predefined procedures and bugs that haven't been squashed yet.

Implementing the interpreter as a bytecode VM increased the scope of the project significantly. While optimized bytecode VMs are theoretically more performant than a tree-walking interpreter, these gains are not yet realized – based on empirical observation, the interpreter currently runs approximately 15 times slower than the tree-walking reference implementation. On a technical note, it would have been better to implement the CST to IR conversion alongside the interpreter. Much of the initial IR code had to be rewritten, and several instances of technical debt linger: IR instructions are not single-purpose, and builtin procedures are represented as a flat enum rather than as dispatchable function pointers, which forces a branching match on every call. The intermediate heap simplified the initial implementation, but made memory bugs more difficult to debug. The heap was initially virtualized. As the IR defines unsafe memory instructions, this did not provide additional safety guarantees but complicated the implementation and made memory debuggers ineffective. Writing comprehensive debugging tools proved invaluable throughout: whenever a bug resisted straightforward reasoning, improving the tooling consistently increased both the speed at which bugs could be resolved and the complexity of bugs that could be tackled.

## 4.2. Outlook

A number of areas remain where this interpreter must be improved before it becomes a compelling alternative to the reference implementation. These fall into three broad categories: semantic correctness, completeness, and performance.

### 4.2.1. Semantic Correctness

Several known semantic gaps exist between this implementation and the reference. Rational number arithmetic is not yet implemented: expressions such as `1/2` currently produce a floating-point result rather than the rational value the reference implementation returns.

The `default` branch in `scan` statements is not currently executed. In the reference implementation, a variable introduced by a `case` branch in a `match` statement does not compare equal to `om`, even when the matched value is om itself. Ordinary variables assigned `om` do compare equal. This implementation does not replicate that distinction: a case-bound variable holding `om` compares equal to om like any other variable. Whether this constitutes an upstream bug in the reference implementation is unclear.

Several test cases expose further correctness issues. Currently, mutating a class member `x` in the constructor, by accessing it through the `this` primitive doesn't mutate the object member if a parameter of the same name `x` exists. In the reference implementation mutation through accessing `this` only fails to propagate to the final value of the object member, if `x` is mutated directly beforehand. Furthermore, in the reference implementation, mutating a static class value also mutates the value for any object of the class that hasn't mutated the value before, while no memory is shared between a class and objects of the class in this implementation. Ambiguous map accessing yields `om` in the reference implemtation, while in this rewrite, the first match is returned instead. Furthermore, in this implementation term evaluation is currently only supported for expressions, but not for statements. As this interpreter doesn't support rational numbers, the factorial operation isn't implemented for fractions.

### 4.2.2. Completeness

A number of predefined procedures remain unimplemented. The most straightforward to add are general-purpose string and collection procedures such as `canonical`, `rational`, `sort`, `shuffle`, `trim`, `toLowerCase`, `toUpperCase`, `startsWith`, `endsWith`, `collect`, `permutations`, `nextPermutation`, `getTerm`, `getScope`, and `isMap`.

Two larger subsystems require more substantial effort. The linear algebra extension – cov-

ering `la_matrix`, `la_solve`, `la_det`, `la_eigenValues`, `la_eigenVectors`, `la_svd`, `la_pseudoInverse`, and `la_cond` – and the plotting and statistical distribution families, which together comprise the majority of the remaining unimplemented surface area. `multiLineMode` and `trace` are out of scope for this implementation, as this interpreter uses different execution and debugging mechanisms; the built-in debugger is arguably more capable than the tracing facilities the reference implementation provides.

### 4.2.3. Performance

The interpreter currently executes the general IR without any optimization passes. A simple expression such as 1+1 currently requires 25 instructions to evaluate where it could be reduced to a single constant. Value propagation and dead branch elimination would therefore improve performance substantially and are the most impactful optimizations to implement.

Beyond optimization passes, several structural changes would improve baseline performance. A dedicated load instruction would eliminate the per-instruction conversion from `IRValue` to `InterpVal`. Making IR instructions single-purpose and replacing the builtin procedure enum with dispatchable function pointers would reduce branching and improve branch predictor effectiveness. Together, these changes should substantially increase the performance of the interpreter, before any JIT compilation is introduced.

Block-level JIT compilation represents another significant performance opportunity. The current dispatch loop interprets each IR instruction by matching it in an if-else chain at runtime. A JIT backend would eliminate this overhead entirely for hot code paths. As discussed in the background chapter, extending the interpreter with a JIT backend would also require registering dynamic unwind information so that exceptions propagate correctly through compiled frames.

# A. Illustrative Examples

## A.1. Reference Counting in jq

jq is a Domain Specific Language (DSL) to query and transform JSON data. A jq expression describes a transformation declaratively. The runtime is responsible for deciding how to apply it efficiently to the underlying data. Internally, jq represents JSON values as reference-counted heap objects, which allows the runtime to avoid unnecessary copies when a value has only one owned, and to perform Copy-On-Write (COW) when a shared value must be mutated. The figure A.2 traces through the execution of the expression A.1 at the C API level, illustrating how reference counts are incremented as values are accessed and how COW is triggered when a shared object needs to be updated.

```
.items[2].price |= . + 10
```

Figure A.1.: jq example expression

```
jv root = jv_parse("{\"items\":[{\"price\":1},{\"price\":2},{\"price\":3}]}"); //
     refcount = 1

jv items = jv_object_get(root, jv_string("items")); // items.refs += 1 (2)
jv item2 = jv_array_get(items, 2); // refcount += 1
jv price = jv_object_get(item2, jv_string("price")); // items[2].refs += 1 (2)
jv new_price = jv_number(jv_number_value(price) + 10); // refcount = 1

/* item[2].refs > 1 -> triggers COW:
      - allocates a fresh object,
      - copies all other fields,
      - items[2].refs -= 1 (1) */
jv new_item2 = jv_object_set(item2, jv_string("price"), new_price);

/* items.refs > 1 -> triggers COW:
      - fresh array buffer,
      - copies all other fields
      - items.refs -= 1 (1) */
jv new_items = jv_array_set(items, 2, new_item2);

/* root.refs == 1
   -> items updated inplace
   -> items.refs -= 1 (0) -> (invalidate underlying memory) */

jv new_root = jv_object_set(root, jv_string("items"), new_items);

printf("updated: %s\n", jv_string_value(jv_dump_string(new_root, 0)));

/* decrement counters of all values
   -> [...].refs -= 1 (0) -> (invalidate underlying memory) */
jv_free(root);
jv_free(items);
jv_free(item2);
jv_free(price);
jv_free(new_price);
jv_free(new_item2);
jv_free(new_items);
jv_free(new_root);
```

Figure A.2.: jq reference counting example

## A.2. Procedure IR Example

Table A.1.: Procedure IR example

```
a := procedure() {
    exit;
};
```

```
_proc0(){
//end block
<bb0>:
  _ := stack_frame_pop();
  return t0;

//start block
<bb1>:
  _ := stack_frame_add();
  _ := exit();
  unreachable;
}

//main
_proc1(){
//end block
<bb0>:
  _ := stack_frame_pop();
  return t0;

//start block
<bb1>:
  _ := stack_frame_add();
  t1 := stack_add("a");
  t2 := _proc0 /* procedure */;
  *t1 := t2;
  goto <bb2>

<bb2>:
  t0 := om;
  goto <bb0>

}
```

Some instructions such as AST nodes or source annotations have been removed for the sake of simplicity. Though it may not be visible in this example, internally the basic blocks and procedures exist in graphs and are connected through edges in the respective graphs. As the procedure `a` is defined in the main procedure, an edge exists between the two procedure nodes. As is apparent in this example not all nodes must necessarily be reachable. Dead-code elimination passes should theoretically remove these redundant blocks.

## A.3. Heap Mutable Borrowing

```
a := 1;
a := 2;
```

```
t1 := stack_add("a");
t2 := 1;              // heap 0 (on immediate heap)
t3 := *t1;            // mutable borrow of stack a
                      // (safe as no other borrows exist
                           )
_ := invalidate(t3); // invalidate prior value of a
                      // a is undefined
                      // -> effectively a noop
*t1 := t2;            // a = 1;
                      // -> a takes ownership of 1 (heap
                           0)
                      // -> heap 0 removed from
                           immediate heap
t4 := 2;              // heap 1 (on immediate heap)
t5 := *t1;            // mutable borrow of stack a
                      // (safe as no other borrows exist
                           )
_ := invalidate(t5); // invalidate prior value of a
                      // -> invalidate heap 0
*t1 := t4;            // a = 2;
                      // -> a takes ownership of 2 (heap
                           1)
                      // -> heap 1 removed from
                           immediate heap
```

Table A.2.: Heap Memory Management IR Example

In this example `t1` is a pointer to the stack value `a`. As will be discussed later, stack values always own the memory they reference. Note that prior to the reassignment of `a` the owned value is invalidated. Invalidation and reassignment must occur as one transaction that cannot cause an exception. Otherwise, use-after-free errors might occur.

## A.4. Stack Closure Example

Table A.3.: Stack Closure IR Example

```
b := 1;
a := closure () {
  print(b);
};
b := 3;
a();
```

```
// main
_proc1(){
<bb2>:
    _ = stack_frame_add(); // create new stack frame
    goto <bb1>

<bb1>:
    t1:<ptr> = stack_add("b");  // push b onto stack
    t2:<ptr> = stack_add("a");  // push a onto stack

    *t1 = 1;  // b := 1
    // capture snapshot of current frame: {b=1, a=om, predefined
          procedures}
    t18:<stack_image> = stack_copy();
    // closure carries snapshot
    // closure is accessible across stack boundaries
    t6:<proc> = procedure_new(_proc0, om, t18, true);
    *t2 = t6;  // a := closure

    // b := 3 only affects the live stack, not the snapshot
    *t1 = 3;
    // call a() passes the snapshot as params[0]
    // retrieve embedded stack image from closure
    t28 = procedure_stack_get(t26);
    t30:<ptr> = &t28;
    // pass snapshot as first param
    _ = list_push(t25, t30);
    t31 = t26(t25);
    _ := invalidate(t31);
    _ := stack_frame_pop();
    return om;
}

// closure
_proc0(){
<bb1>:
    // create new stack frame
    _ := stack_frame_add();
    t1:<ptr> = params[0];
    t2:<stack_image> = *t1;
    // restore snapshot into current frame: b=1 visible here
    _ = stack_frame_restore(t2);
    // resolves to b=1 from restored snapshot
    t5:<ptr> = stack_get_or_new("b");
    // resolves to predefined procedure print
    // copy of print exists in current stack frame
    t6:<ptr> = stack_get_or_new("print");
    t4 := list_new();
    _ := list_push(t4, t5);
    t12 = t6(t4);
    // invalidate restored snapshot
    _ := stack_frame_pop();
    return om;
}
```

When `proc1` begins, `stack_copy()` captures a snapshot of all currently reachable stack entries after `b:=1` is assigned, including `b:=1`, `a:=om`, and all predefined procedures. The snapshot is embedded into the closure with a cross-frame flag. When `b` is subsequently reassigned to $3$, the snapshot is unaffected. When `a()` is called, the snapshot is retrieved and passed as the first parameter. Inside `proc0`, `stack_frame_restore()` pushes copies of all snapshot entries into the current frame, making `b:=1` and `print` visible as ordinary stack variables. When the closure returns, `stack_frame_pop()` discards all restored entries at once.

## A.5. Class IR Example

Table A.4.: Class IR Example

```
class a() {
  sum := procedure(
       other) {
    return 2;
  };
}
b := a();
print(b + 1);
```

```
// constructor
_proc1(){
<bb0>:
    _ = stack_frame_add();
    t0:<ptr> = stack_add("sum");
    *t0 = procedure_new(_proc0, om, true);
    t1:<list> = stack_frame_save();
    t2:<obj> = object_new("a", t1);
    return t2;
}

// static procedure (empty static block)
_proc2(){
<bb0>:
    _ = stack_frame_add();
    t0:<list> = stack_frame_save();
    return t0;
}

// main
_proc3(){
<bb1>:
    _ = stack_frame_add();
    t1:<ptr> = stack_add("b");
    _ = class_add("a", _proc2, _proc1);
    t4:<ptr> = stack_get_or_new("a");
    t5 = *t4;
    t10 = t5(t3);         // call constructor
    *t1 = t10;            // b := object{sum=proc}
    t23 = *t1;            // load b
    t31 = object_get(t23, "sum"); // overload for +
    t29:<ptr> = &t23;
    t36:<list> = list_new();
    _ = list_push(t36, &t24);     // push 1 as argument
    _ = stack_frame_add();
    _ = stack_alias("this", t29, true);
    // alias all members of b by name (cross-frame)
    t44 = t33(t36);       // call sum(1) -> 2
    _ = stack_frame_pop();
    // print(2) omitted
    return om;
}
```

The class is registered via `class_add`, giving it priority over the stack. Calling `a()` invokes the

constructor, which saves its frame as an object containing `sum`. When `b+1` is evaluated, `sum` is found as the overload for `+`, a new frame is pushed with `b` aliased as `this` and all its members aliased by name, and `sum(1)` is called. The frame is torn down on return.

# B. Supported Features

## B.1. Predefined Procedure Support

Table B.1.: Predefined Procedure Support

| **Procedure** | **Status** |
|---|---|
| `abort` | Supported |
| `abs` | Supported |
| `appendFile` | Supported |
| `arb` | Divergent — always returns the first element; the reference implementation returns the first or last element depending on the input |
| `args` | Supported |
| `ask` | Supported |
| `assert` | Supported |
| `cacheStats` | Unsupported |
| `canonical` | Unsupported |
| `ceil` | Supported |
| `char` | Supported |
| `clearCache` | Supported |
| `collect` | Unsupported |
| `compare` | Supported |
| `cos`, `cosh` | Supported |
| `deleteFile` | Supported |
| `domain` | Supported |
| `double` | Supported |
| `endsWith`, `startsWith` | Unsupported |

continued on next page

Table B.1 – continued

| Procedure | Status |
|---|---|
| `eval` | Supported |
| `evalTerm` | Supported |
| `execute` | Supported |
| `exp` | Supported |
| `fct` | Supported |
| `first`, `last` | Supported |
| `floor` | Supported |
| `from` | Divergent — always returns the first element; the reference implementation returns the first or last element depending on the input |
| `fromB`, `fromE` | Supported |
| `get` | Divergent — supports readline history navigation via arrow keys; the reference implementation does not |
| `getOsID` | Supported |
| `getScope` | Unsupported |
| `getTerm` | Unsupported |
| `int` | Supported |
| `isBoolean` | Supported |
| `isDouble` | Supported |
| `isError` | Unsupported |
| `isInfinite` | Supported |
| `isInteger` | Supported |
| `isList` | Supported |
| `isMap` | Unsupported |
| `isNumber` | Supported |
| `isObject` | Supported |
| `isPrime`, `isProbablePrime` | Supported |
| `isRational` | Unsupported |
| `isSet` | Supported |
| `isString` | Supported |

continued on next page

Table B.1 – continued

| Procedure | Status |
|---|---|
| isTerm | Supported |
| isVariable | Supported |
| join | Supported |
| la_cond | Unsupported |
| la_det | Unsupported |
| la_eigenValues, la_eigenVectors | Unsupported |
| la_matrix | Unsupported |
| la_pseudoInverse | Unsupported |
| la_solve | Unsupported |
| la_svd | Unsupported |
| load, loadLibrary | Supported |
| log, log10, log1p | Supported |
| logo | Supported |
| makeTerm | Supported |
| matches | Supported |
| mathConst | Supported |
| max, min | Supported |
| multiLineMode | Out of scope |
| nextPermutation | Unsupported |
| nextProbablePrime | Unsupported |
| nPrint, nPrintErr | Supported |
| now | Supported |
| parse | Supported |
| parseStatements | Supported |
| permutations | Unsupported |
| plot_addBarChart | Unsupported |
| plot_addBullets | Unsupported |
| plot_addGraph | Unsupported |
| plot_addLabel | Unsupported |
| plot_addListGraph | Unsupported |

continued on next page

Table B.1 – continued

| Procedure | Status |
|---|---|
| plot_addParamGraph | Unsupported |
| plot_addPieChart | Unsupported |
| plot_createCanvas | Unsupported |
| plot_defineTitle | Unsupported |
| plot_exportCanvas | Unsupported |
| plot_labelAxis | Unsupported |
| plot_legendVisible | Unsupported |
| plot_modScale, plot_modScaleType | Unsupported |
| plot_modSize | Unsupported |
| plot_removeGraph | Unsupported |
| pow | Supported |
| print, printErr | Supported |
| raise, throw | Supported |
| random, rnd | Supported |
| range | Supported |
| rational | Unsupported |
| read | Supported |
| readFile | Supported |
| replace, replaceFirst | Supported |
| resetRandom | Divergent — Doesn't do anything. Only true random number generators are supported. |
| reverse | Supported |
| round | Supported |
| run | Supported |
| signum | Supported |
| sin, sinh | Supported |
| sleep | Supported |
| sort, shuffle | Unsupported |
| split | Supported |
| sqrt | Divergent — not implemented for all types |

continued on next page

Table B.1 – continued

| Procedure | Status |
|---|---|
| stat_beta, stat_betaCDF | Unsupported |
| stat_beta_plot, stat_betaCDF_plot | Unsupported |
| stat_cauchy, stat_cauchyCDF | Unsupported |
| stat_cauchy_plot, stat_cauchyCDF_plot | Unsupported |
| stat_chiSquared, stat_chiSquaredCDF | Unsupported |
| stat_chiSquared_plot, stat_chiSquaredCDF_plot | Unsupported |
| stat_exponential, stat_exponentialCDF | Unsupported |
| stat_exponential_plot, stat_exponentialCDF_plot | Unsupported |
| stat_fisher, stat_fisherCDF | Unsupported |
| stat_fisher_plot, stat_fisherCDF_plot | Unsupported |
| stat_gamma, stat_gammaCDF | Unsupported |
| stat_gamma_plot, stat_gammaCDF_plot | Unsupported |
| stat_levy, stat_levyCDF | Unsupported |
| stat_levy_plot, stat_levyCDF_plot | Unsupported |

continued on next page

Table B.1 – continued

| Procedure | Status |
|---|---|
| stat_logNormal, stat_logNormalCDF | Unsupported |
| stat_logNormal_plot, stat_logNormalCDF_plot | Unsupported |
| stat_normal, stat_normalCDF | Unsupported |
| stat_normal_plot, stat_normalCDF_plot | Unsupported |
| stat_pareto, stat_paretoCDF | Unsupported |
| stat_pareto_plot, stat_paretoCDF_plot | Unsupported |
| stat_student, stat_studentCDF | Unsupported |
| stat_student_plot, stat_studentCDF_plot | Unsupported |
| stat_weibull, stat_weibullCDF | Unsupported |
| stat_weibull_plot, stat_weibullCDF_plot | Unsupported |
| str | Supported |
| tan, tanh | Supported |
| toLowerCase, toUpperCase | Unsupported |
| trace | Out of scope |
| trim | Unsupported |
| ulp | Supported |
| writeFile | Supported |

# C. Test Results

Table C.1.: SetlX Logic Lecture Test Results

| **Program** | **Result** |
|---|---|
| allValuations.stlx | ✓ |
| append-match.stlx | ✓ |
| append.stlx | ✓ |
| arb.stlx | ✓ |
| buggy-function.stlx | ✓ |
| check-equivalence.stlx | ✓ |
| completeness.stlx | ✓ |
| count.stlx | ✓ |
| davis-putnam.stlx | ✓ |
| diff.stlx | ✓ |
| evaluate.stlx | ✓ |
| evaluateOld.stlx | ✓ |
| fibonacci-combinatorics.stlx | ✓ |
| find-path.stlx | ✓ |
| find-perfect.stlx | ✓ |
| fixpoint.stlx | ✓ |
| fol-evaluate.stlx | ✓ |
| from.stlx | ordering |
| function.stlx | serialize (rational numbers as floats) |
| ggt-fast.stlx | ✓ |
| ggt-loop.stlx | ✓ |
| graph-2-dot.stlx | ✓ |
| gruppen.stlx | ✓ |
| hanoi-frame.stlx | ✓ |
| knf.stlx | ✓ |

| Program | Result |
|---|---|
| legendre.stlx | ✓ |
| min-sort.stlx | ✓ |
| mySort.stlx | ✓ |
| parse-propositional.stlx | ✓ |
| path-cyclic.stlx | ✓ |
| path.stlx | ✓ |
| poker-triple.stlx | ✓ |
| power.stlx | ✓ |
| primes-eratosthenes.stlx | ✓ |
| primes-for.stlx | ✓ |
| primes-forall.stlx | ✓ |
| primes-recursive.stlx | ✓ |
| primes-sieve.stlx | ✓ |
| primes-slim.stlx | ✓ |
| primes-tuple.stlx | ✓ |
| primes-while.stlx | ✓ |
| queens-backtrack.stlx | ✓ |
| queens-search.stlx | ✓ |
| queens.stlx | ✓ |
| quine.stlx | ✓ |
| quone.stlx | ✓ |
| reverse.stlx | ✓ |
| simple-tuple.stlx | ✓ |
| simple.stlx | ✓ |
| solve.stlx | ✓ |
| sort.stlx | ✓ |
| stops.stlx | ✓ |
| sum-recursive.stlx | ✓ |
| sum.stlx | ✓ |
| switch.stlx | ✓ |
| tautology.stlx | ✓ |
| tautologyOld.stlx | ✓ |
| test-completeness.stlx | ✓ |

| Program | Result |
|---|---|
| test-davis-putnam.stlx | ✓ |
| test-knf.stlx | ✓ |
| transitive-closure.stlx | ✓ |
| transitive-recursive.stlx | ✓ |
| tunnel-frame.stlx | ✓ |
| turing.stlx | ✓ |
| watson.stlx | ✓ |
| wolf-goat-cabbage.stlx | ✓ |
| zebra-frame.stlx | ✓ |

Table C.2.: SetlX Interpreter Test Results

| Program | Result |
|---|---|
| aStar.stlx | ✓ |
| aStar_hanoi.stlx | serialize (rational numbers as floats) |
| aStar_hanoi_exact.stlx | ✓ |
| check-equivalence.stlx | ✓ |
| complex.stlx | ✗ unimplemented: `atan2`, `hypot` |
| cyk.stlx | ✗ wrong result |
| dempster-shafer.stlx | ✓ |
| dfa-2-regexp.stlx | ✗ wrong result |
| diff.stlx | serialize (rational numbers as floats) |
| differentiate.stlx | ✗ `canonical` undefined |
| earley-parser.stlx | ✗ or undefined for type |
| erlang.stlx | ✓ |
| euler.stlx | ✗ `nDecimalPlaces` undefined |
| evaluate.stlx | ✓ |
| exam.stlx | ✗ scan branch on non-boolean expression |
| fibonacci.stlx | ✗ `cacheStats` undefined |
| finite-automaton.stlx | ✓ |
| insertion-sort.stlx | ✓ |
| knapsack-cached.stlx | ✗ does not terminate |
| kosarajuSCC.stlx | ✓ |

| Program | Result |
|---|---|
| llk.stlx | ✗ wrong result |
| minCut.stlx | ✓ |
| minimize.stlx | ✓ |
| montecarlo.stlx | ✗ wrong result |
| nda.stlx | ✓ |
| nfa2dfa.stlx | ✓ |
| parsing_lr-table-generator.stlx | ✗ wrong code bug |
| parsing_shift-reduce-parser.stlx | ✗ `tokenizeString` undefined |
| parsing_slr-table-generator.stlx | ✗ wrong code bug |
| plot.stlx | serialize (rational as float, procedure formatting, semicolon), float precision |
| primes-sieve.stlx | ✓ |
| primes-tuple.stlx | ✓ |
| queens.stlx | ✗ does not terminate |
| queens-backtrack.stlx | ✓ |
| regression.stlx | serialize (different floating point precision) |
| tautologySimple.stlx | serialize (parse error formatted differently) |
| test-equivalence.stlx | ✓ |
| trampoline.setlx | does not terminate (just like the reference implementation) |
| xmlscanner.stlx | ✓ |
| zebra-brute-force.stlx | ✗ `permutations` undefined |
| *animation_testcode* | |
| aStar_hanoi_exact_animation.stlx | ✓ |
| hanoi_animation.stlx | ✗ unimplemented: `gfx_setCanvasSize` |
| *converted_Setl2_code* | |
| 2.12_Printing_the_elements_of_a_set.stlx | ordering |
| 2.13_More_Operators_on_lists_and_sets.stlx | ✓ |
| 2.18_A_n_infinite_loop.stlx | ✓ |
| 2.8_non-functional_binary_relations.stlx | ✗ wrong result |
| aufgaben_zur_gdi.stlx | ✓ |
| case.stlx | ✓ |
| closure.stlx | ✓ |

| **Program** | **Result** |
|---|---|
| function.stlx | serialize (rational numbers as floats) |
| ggt.stlx | ✓ |
| ggt2.stlx | ✓ |
| hanoi-slim.stlx | ✓ |
| hanoi.stlx | ✓ |
| hanoi2.stlx | ✓ |
| hello.stlx | ✓ |
| kgv.stlx | ✓ |
| kgv2.stlx | ✓ |
| klausur_aufgabe_power.stlx | ✓ |
| klausur_aufgaben.stlx | ✓ |
| knf.stlx | ✓ |
| kokosnuss.stlx | serialize (rational numbers as floats) |
| min-sort.stlx | ✓ |
| pfefferkoerner.stlx | ✓ |
| poker-tripple.stlx | ✓ |
| poker.stlx | ✓ |
| primes-eratosthenes-fast.stlx | ✓ |
| primes-eratosthenes.stlx | ✓ |
| primes-forall.stlx | ✓ |
| primes-set.stlx | ✓ |
| primes-sieve.stlx | ✓ |
| primes-while.stlx | ✓ |
| primes.stlx | ✓ |
| simple.stlx | ✓ |
| solve.stlx | ✓ |
| sum.stlx | ✓ |
| tautology.stlx | ✗ wrong result |
| variables.stlx | ✗ wrong code bug |
| watson.stlx | ✓ |
| weinhaendler.stlx | ✓ |
| wolf-ziege-kohl.stlx | ✓ |
| *performance_test_code* | |

| Program | Result |
|---|---|
| ackermann_function.stlx | ✗ stack overflow |
| count_the_coins.stlx | ✗ out of memory |
| cryptoarithmetic.stlx | ✗ wrong result |
| erlang.stlx | ✓ |
| euler.stlx | ✓ |
| euler01.stlx | ✓ |
| euler02.stlx | ✓ |
| euler03.stlx | serialize (rational number as float) |
| euler04.stlx | ✗ wrong result |
| euler05.stlx | ✓ |
| euler06.stlx | serialize (rational numbers as floats) |
| euler07.stlx | ✓ |
| euler08.stlx | ✓ |
| euler09.stlx | ✗ wrong output |
| euler10.stlx | ✓ |
| euler11.stlx | ✓ |
| euler12.stlx | ✗ wrong output |
| euler14a1.stlx | ✓ |
| euler14a2.stlx | ✗ does not terminate |
| euler14b.stlx | ✗ does not terminate |
| euler15.stlx | serialize (rational numbers as floats) |
| euler16.stlx | ✗ plus not defined for type (rational numbers) |
| euler17.stlx | ✓ |
| euler18.stlx | ✓ |
| euler19.stlx | ✓ |
| euler20.stlx | ✓ |
| euler21.stlx | ✓ |
| euler23.stlx | ✗ does not terminate |
| euler24.stlx | ✗ `permutations` undefined |
| euler25.stlx | ✓ |
| euler26.stlx | ✓ |
| euler27.stlx | ✗ does not terminate |
| euler27b.stlx | ✓ |

| Program | Result |
|---|---|
| euler28.stlx | serialize (rational number as float) |
| euler29.stlx | ✗ wrong result |
| euler30.stlx | ✗ does not terminate |
| euler31.stlx | ✓ |
| euler32.stlx | ✗ does not terminate |
| euler33.stlx | serialize (rational numbers as floats) |
| euler34.stlx | ✗ does not terminate |
| euler35.stlx | ✗ does not terminate |
| euler35b.stlx | ✓ |
| euler36.stlx | ✗ does not terminate |
| euler37.stlx | ✗ does not terminate |
| euler38.stlx | ✗ wrong result |
| euler39a.stlx | ✗ does not terminate |
| euler39b.stlx | ✗ does not terminate (out of memory) |
| euler40.stlx | ✗ does not terminate |
| euler41.stlx | ✗ wrong result |
| euler43a1.stlx | ✗ does not terminate |
| euler43a2.stlx | ✗ `permutations` undefined |
| euler43b.stlx | ✗ `nextPermutation` undefined |
| euler44.stlx | ✓ |
| euler45.stlx | serialize (rational numbers as floats) |
| euler46.stlx | ✗ wrong result |
| euler47.stlx | ✗ does not terminate (out of memory) |
| euler48.stlx | ✗ wrong result |
| euler49.stlx | ✓ |
| euler49b.stlx | ✗ `permutations` undefined |
| factorial_list.stlx | ✓ |
| factorial_loop.stlx | ✓ |
| factorial_native.stlx | ✓ |
| factorial_native_print.stlx | ✓ |
| fast-crypto.stlx | ✗ `compileWord` undefined |
| lr-table-generator.stlx | ✗ iterator undefined for type |
| minCut.stlx | ✓ |

| Program | Result |
|---|---|
| nqueens1.stlx | ✗ `permutations` undefined |
| nqueens2.stlx | ✗ `nextPermutation` undefined |
| nqueens3.stlx | ✗ does not terminate |
| nqueens4.stlx | ✗ `permutations` undefined |
| nqueens5.stlx | ✗ does not terminate |
| nqueens6.stlx | ✗ `nextPermutation` undefined |
| perfect.stlx | ✓ |
| pocket-pair.stlx | ✗ `resetRandom` undefined |
| send_more_money_1a.stlx | ✓ |
| send_more_money_1b.stlx | ✓ |
| send_more_money_1c.stlx | ✓ |
| send_more_money_2.stlx | ✗ `permutations` undefined |
| send_more_money_3.stlx | ✗ does not terminate |
| send_more_money_4.stlx | ✗ `permutations` undefined |
| sorting_bench.stlx | ✗ does not terminate |
| tic-tac-toe.stlx | ✗ wrong result |
| tsp.stlx | ✓ |
| zebra.stlx | ✗ does not terminate |
| *plotting_test_code* | |
| addBarChart.stlx | ✗ `plot_createCanvas` undefined |
| addBoxChart.stlx | ✗ `plot_createCanvas` undefined |
| addBullets.stlx | ✗ `plot_createCanvas` undefined |
| addGraph.stlx | ✗ `plot_createCanvas` undefined |
| addLabel.stlx | ✗ `plot_createCanvas` undefined |
| addListGraph.stlx | ✗ `plot_createCanvas` undefined |
| addParamGraph.stlx | ✗ `plot_createCanvas` undefined |
| addPieChart.stlx | ✗ `plot_createCanvas` undefined |
| circle.stlx | ✗ `plot_createCanvas` undefined |
| convergence-to-normal.stlx | ✗ `resetRandom` undefined |
| defineTitle.stlx | ✗ `plot_createCanvas` undefined |
| exportCanvas.stlx | ✗ `plot_createCanvas` undefined |
| fathers-and-sons.stlx | ✗ `plot_createCanvas` undefined |
| hypotrochoid.stlx | ✗ `plot_createCanvas` undefined |

| Program | Result |
|---|---|
| labelAxis.stlx | ✗ `plot_createCanvas` undefined |
| legendVisible.stlx | ✗ `plot_createCanvas` undefined |
| modScale.stlx | ✗ `plot_createCanvas` undefined |
| modScaleType.stlx | ✗ `plot_createCanvas` undefined |
| modSize.stlx | ✗ `plot_createCanvas` undefined |
| parabola.stlx | ✗ `plot_createCanvas` undefined |
| race-and-incarceration.stlx | ✗ `plot_createCanvas` undefined |
| removeGraph.stlx | ✗ `plot_createCanvas` undefined |
| world-population.stlx | ✗ `plot_createCanvas` undefined |
| *simple_examples* | |
| backtrack.stlx | ✓ |
| builtInFunctions.stlx | ✗ `resetRandom` undefined |
| compare.stlx | ordering, serialize (rational as float, predefined procedure) |
| crazyParserSyntaxErrorCheck.stlx | ✓ |
| dot-product.stlx | ✓ |
| errorOutput.stlx | ✓ |
| factorial.stlx | ✓ |
| fileAPI.stlx | ✗ does not terminate (`deleteFile` froze) |
| formatedString.stlx | ✓ |
| formatedStringError.stlx | ✓ |
| match.stlx | ✗ `canonical` undefined |
| objectOrientation.stlx | ✗ factor undefined for type |
| parseAndEval.stlx | ✗ `canonical` undefined |
| prologStyleListMatch.stlx | ✓ |
| regex.stlx | ✗ wrong result |
| simpleClosure.stlx | serialize (rational as float, predefined procedure) |
| tryCatch.stlx | ✗ `isError` undefined |
| *simple_examples/gfx_addon* | |
| addPlayPauseButton.stlx | ✗ `gfx_addPlayPauseButton` undefined |
| addSpeedSlider.stlx | ✗ `gfx_addSpeedSlider` undefined |
| arc.stlx | ✗ `gfx_arc` undefined |

| Program | Result |
|---|---|
| circle.stlx | ✗ `gfx_circle` undefined |
| clear.stlx | ✗ `gfx_clear` undefined |
| ellipse.stlx | ✗ `gfx_ellipse` undefined |
| filledCircle.stlx | ✗ `gfx_filledCircle` undefined |
| filledEllipse.stlx | ✗ `gfx_filledEllipse` undefined |
| filledPolygon.stlx | ✗ `gfx_filledPolygon` undefined |
| getFont.stlx | ✗ `gfx_getFont` undefined |
| getPenColor.stlx | ✓ |
| getPenRadius.stlx | ✗ `gfx_getPenRadius` undefined |
| hasNextKeyTyped.stlx | ✗ `gfx_hasNextKeyTyped` undefined |
| isKeyPressed.stlx | ✗ `gfx_isKeyPressed` undefined |
| isPaused.stlx | ✗ `gfx_isPaused` undefined |
| line.stlx | ✗ `gfx_line` undefined |
| mouseX.stlx | ✗ `gfx_mouseX` undefined |
| mouseY.stlx | ✗ `gfx_mouseY` undefined |
| nextKeyTyped.stlx | ✗ `gfx_nextKeyTyped` undefined |
| picture.stlx | ✗ `gfx_picture` undefined |
| point.stlx | ✗ `gfx_point` undefined |
| polygon.stlx | ✗ `gfx_polygon` undefined |
| rectangle.stlx | ✗ `gfx_rectangle` undefined |
| setCanvasSize.stlx | ✗ `gfx_setCanvasSize` undefined |
| setFont.stlx | ✗ `gfx_setFont` undefined |
| setMessage.stlx | ✗ `gfx_setMessage` undefined |
| setPaused.stlx | ✗ `gfx_setPaused` undefined |
| setPenColor.stlx | ✗ `gfx_setPenColor` undefined |
| setPenColorRGB.stlx | ✗ `gfx_setPenColorRGB` undefined |
| setPenRadius.stlx | ✗ `gfx_setPenRadius` undefined |
| setScale.stlx | ✗ `gfx_setScale` undefined |
| setXScale.stlx | ✗ `gfx_setXScale` undefined |
| setYScale.stlx | ✗ `gfx_setYScale` undefined |
| show.stlx | ✗ `gfx_show` undefined |
| square.stlx | ✗ `gfx_square` undefined |
| test-gfx.stlx | ✗ `gfx_setCanvasSize` undefined |

| Program | Result |
|---|---|
| text.stlx | ✗ `gfx_text` undefined |
| textLeft.stlx | ✗ `gfx_textLeft` undefined |
| textRight.stlx | ✗ `gfx_textRight` undefined |
| *stat_test_code* | |
| stat_beta.stlx | ✗ `stat_beta` undefined |
| stat_betaCDF.stlx | ✗ `stat_betaCDF` undefined |
| stat_cauchy.stlx | ✗ `stat_cauchy` undefined |
| stat_cauchyCDF.stlx | ✗ `stat_cauchyCDF` undefined |
| stat_chiSquared.stlx | ✗ `stat_chiSquared` undefined |
| stat_chiSquaredCDF.stlx | ✗ `stat_chiSquaredCDF` undefined |
| stat_exponential.stlx | ✗ `stat_exponential` undefined |
| stat_exponentialCDF.stlx | ✗ `stat_exponentialCDF` undefined |
| stat_fisher.stlx | ✗ `stat_fisher` undefined |
| stat_fisherCDF.stlx | ✗ `stat_fisherCDF` undefined |
| stat_gamma.stlx | ✗ `stat_gamma` undefined |
| stat_gammaCDF.stlx | ✗ `stat_gammaCDF` undefined |
| stat_levy.stlx | ✗ `stat_levy` undefined |
| stat_levyCDF.stlx | ✗ `stat_levyCDF` undefined |
| stat_logNormal.stlx | ✗ `stat_logNormal` undefined |
| stat_logNormalCDF.stlx | ✗ `stat_logNormalCDF` undefined |
| stat_normal.stlx | ✗ `stat_normal` undefined |
| stat_normalCDF.stlx | ✗ `stat_normalCDF` undefined |
| stat_pareto.stlx | ✗ `stat_pareto` undefined |
| stat_paretoCDF.stlx | ✗ `stat_paretoCDF` undefined |
| stat_student.stlx | ✗ `stat_student` undefined |
| stat_studentCDF.stlx | ✗ `stat_studentCDF` undefined |
| stat_weibull.stlx | ✗ `stat_weibull` undefined |
| stat_weibullCDF.stlx | ✗ `stat_weibullCDF` undefined |

# D. LALR(1) SetlX Grammar

```
(* Whitespace and comments are skipped *)

Block
    : Statement*
    ;

Statement
    : ClassDecl
    | IfDecl
    | SwitchDecl
    | MatchDecl
    | ScanDecl
    | ForDecl
    | WhileDecl
    | DoWhileDecl
    | TryCatchDecl
    | CheckDecl
    | ReturnDecl
    | AssignDecl ';'
    | AssignModDecl ';'
    | Expr ';'
    | 'backtrack' ';'
    | 'break' ';'
    | 'continue' ';'
    | 'exit' ';'
    ;

ClassDecl
    : 'class' Ident '(' ProcedureParams ')'
      '{' Block ClassStaticDecl? '}' ';'?
    ;

ClassStaticDecl
    : 'static' '{' Block '}'
    ;

IfDecl
    : 'if' '(' Expr ')' '{' Block '}'
      ElifDecl*
      ElseDecl?
    ;

ElifDecl
    : 'else' 'if' '(' Expr ')' '{' Block '}'
    ;

ElseDecl
    : 'else' '{' Block '}'
    ;

SwitchDecl
    : 'switch' '{' SwitchCase* SwitchDefault? '}'
    ;

SwitchCase
    : 'case' Expr ':' Block
    ;

SwitchDefault
    : 'default' ':' Block
    ;

MatchDecl
    : 'match' '(' Expr ')'
      '{' MatchBranch+ MatchDefault? '}'
    ;

MatchDefault
    : 'default' ':' Block
    ;

MatchBranch
    : MatchCaseBranch
    | MatchRegexBranch
    ;

MatchCaseBranch
    : 'case' Expr (',' Expr)* PipeExpr? ':' Block
    ;

MatchRegexBranch
    : 'regex' Expr MatchRegexBranchAs? PipeExpr? ':' Block
    ;

MatchRegexBranchAs
    : 'as' Expr
    ;

ScanDecl
    : 'scan' '(' Expr ')' ScanUsing? '{' MatchRegexBranch+ '}'
    ;

ScanUsing
    : 'using' Ident
    ;

ForDecl
    : 'for' '(' IteratorChain PipeExpr? ')' '{' Block '}'
```

```
    ;

WhileDecl
    : 'while' '(' Expr ')' '{' Block '}'
    ;

DoWhileDecl
    : 'do' '{' Block '}' 'while' '(' Expr ')' ';'
    ;

TryCatchDecl
    : 'try' '{' Block '}' CatchBranches
    ;

CatchBranches
    : CatchCondBranch* CatchFinBranch?
    ;

CatchFinBranch
    : 'catch' '(' Ident ')' '{' Block '}'
    ;

CatchCondBranch
    : CatchLngBranch
    | CatchUsrBranch
    ;

CatchLngBranch
    : 'catchLng' '(' Ident ')' '{' Block '}'
    ;

CatchUsrBranch
    : 'catchUsr' '(' Ident ')' '{' Block '}'
    ;

CheckDecl
    : 'check' '{' Block '}' CheckAfterBacktrack?
    ;

CheckAfterBacktrack
    : 'afterBacktrack' '{' Block '}'
    ;

ReturnDecl
    : 'return' Expr? ';'
    ;

AssignDecl
    : Accessible ':=' AssignSource
    ;

AssignSource
    : AssignDecl
    | Expr
    ;

AssignModDecl
    : Accessible AssignModKind Expr
    ;

AssignModKind
    : '+='
    | '-='
    | '*='
    | '/='
    | '\='
    | '%='
    ;

ExprList
    : Expr (',' Expr)*
    ;

Expr
    : Lambda
    | SetEqual
    ;

Lambda
    : LambdaParams LambdaIsClosure Expr
    ;

LambdaParams
    : Ident
    | BrackCollection
    ;

LambdaIsClosure
    : '|=>'
    | '|->'
    ;

SetEqual
    : Implication (SetEqualOp Implication)?
    ;

SetEqualOp
    : '<==>'
    | '<!=>'
    ;

Implication
    : Disjunction ('=>' Implication)?
    ;

Disjunction
    : Conjunction ('||' Conjunction)*
    ;

Conjunction
    : Comparison ('&&' Comparison)*
    ;

Comparison
    : Sum (ComparisonOp Sum)?
    ;

ComparisonOp
    : '=='
    | '!='
    | '<'
    | '<='
```

```
    | '>'
    | '>='
    | 'in'
    | 'notin'
    ;

Sum
    : Product (SumOp Product)*
    ;

SumOp
    : '+'
    | '-'
    ;

Product
    : Reduce (ProductOp Reduce)*
    ;

ProductOp
    : '*'
    | '/'
    | '\'
    | '%'
    | '><'
    ;

Reduce
    : PrefixOperation (ReduceOp PrefixOperation)*
    ;

ReduceOp
    : '+/'
    | '*/'
    ;

PrefixOperation
    : Factor ('**' PrefixOperation)?
    | PrefixUnaryOp PrefixOperation
    ;

PrefixUnaryOp
    : '+/'
    | '*/'
    | '#'
    | '-'
    ;

Factor
    : '!' Factor
    | QuantKind '(' IteratorChain '|' Expr ')'
    | AccessibleProcedure '!'?
    ;

QuantKind
    : 'exists'
    | 'forall'
    ;

AccessibleProcedure
    : Procedure
    | Accessible
    ;

Accessible
    : AccessibleHead AccessibleBodyElem*
    | Value
    ;

AccessibleHead
    : '(' Expr ')'
    | Ident
    | Call
    ;

AccessibleBodyElem
    : '.' Ident
    | '.' Call
    | BrackCollection
    | CurlyCollection
    ;

AccessibleList
    : Accessible (',' Accessible)*
    ;

Call
    : Ident '(' ProcedureCallParams? ')'
    ;

ProcedureCallParams
    : Expr (',' Expr)* ',' '*' Expr
    | Expr (',' Expr)*
    | '*' Expr
    ;

Procedure
    : ProcedureKind '(' ProcedureParams ')' '{' Block '}'
    ;

ProcedureKind
    : 'procedure'
    | 'cachedProcedure'
    | 'closure'
    ;

ProcedureParams
    : ProcedureParam (',' ProcedureParam)* (',' DefaultParam)* (',' ListParam)?
    | DefaultParam (',' DefaultParam)* (',' ListParam)?
    | ListParam?
    ;

ProcedureParam
    : 'rw'? Ident
    ;

DefaultParam
    : Ident ':=' Expr
    ;

ListParam
```

```
    : '*' Ident
    ;

PipeExpr
    : '|' Expr
    ;

TermName
    : Term
    | TTerm
    ;

TermExpr
    : TermName '(' Expr (',' Expr)* ')'
    | TermName '(' ')'
    ;

Value
    : Collection
    | StringRule
    | Literal
    | Matrix
    | Vector
    | TermExpr
    | AtomicValue
    | '_'
    ;

Collection
    : CurlyCollection
    | BrackCollection
    ;

ExtendedCollection
    : Expr '..' Expr
    | Expr (',' Expr)* PipeExpr?
    ;

BrackCollection
    : '[' Expr ',' ExtendedCollection ']'
    | '[' Expr '..' Expr? ']'
    | '[' '..' Expr ']'
    | '[' Expr ':' IteratorChain PipeExpr? ']'
    | '[' Expr PipeExpr ']'
    | '[' Expr? ']'
    ;

CurlyCollection
    : '{' Expr ',' ExtendedCollection '}'
    | '{' Expr '..' Expr? '}'
    | '{' '..' Expr '}'
    | '{' Expr ':' IteratorChain PipeExpr? '}'
    | '{' Expr PipeExpr '}'
    | '{' Expr? '}'
    ;

VecNumber
    : Number
    | Double
    ;

Vector
    : '<<' ('-'? VecNumber ('/' Number)?)+ '>>'
    ;

Matrix
    : '<<' Vector+ '>>'
    ;

AtomicValue
    : Number
    | Double
    | 'om'
    | 'true'
    | 'false'
    ;

Iterator
    : Accessible 'in' Expr
    ;

IteratorChain
    : Iterator (',' Iterator)*
    ;

Ident
    : [a-zA-Z][a-zA-Z_0-9]*
    ;

Term
    : '@' [a-zA-Z][a-zA-Z_0-9]*
    ;

TTerm
    : '@@@' [a-zA-Z][a-zA-Z_0-9]*
    ;

Number
    : '0' | [1-9][0-9]*
    ;

Double
    : ([0-9]+)? '.' [0-9]+ ([eE] [+-]? [0-9]+)?
    ;

StringRule
    : '"' ('\\' . | ~['"'\\])* '"'
    ;

Literal
    : '\'' ('\'\'' | ~['\''])* '\''
    ;
```

Listing D.1: LALR SetlX grammar

## Hinweis zum Einsatz von KI-Werkzeugen

Im Rahmen dieser Studienarbeit wurden KI-gestützte Sprachmodelle unterstützend eingesetzt. ChatGPT (OpenAI), Claude (Anthropic) sowie Kimi K2 (Moonshot AI) wurden für die Code-Generierung, die sprachliche Überarbeitung von Textabschnitten sowie zum Korrekturlesen verwendet. Sämtlicher generierter Code wurde vom Autor auf Korrektheit geprüft und eigenständig angepasst. Die inhaltliche Auseinandersetzung mit dem Thema, die Argumentation sowie alle fachlichen Schlussfolgerungen sind eigene Leistung des Autors.

# Bibliography


[1] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. Pearson Education, Inc, 2004. ISBN: 0-201-10088-6.

[2] Thorsten Ball. *Writing an Interpreter in Go*. Self-published, 2016. ISBN: 978-3-982016-00-8.

[3] Joshua Barretto. *ariadne Source Code*. URL: https://codeberg.org/zesterer/ariadne (visited on 04/08/2026).

[4] Stephen Chong. *CS153: Compilers Lecture 17: Control Flow Graph and Data Flow Analysis*. 2018. URL: https://groups.seas.harvard.edu/courses/cs153/2018fa/lectures/Lec17-CFG-dataflow.pdf (visited on 04/08/2026).

[5] Phillip B. Gibbons. *Lecture 5: Introduction to Data Flow Analysis*. 2016. URL: https://www.cs.cmu.edu/afs/cs/academic/class/15745-s16/www/lectures/L5-Intro-to-Dataflow.pdf (visited on 04/08/2026).

[6] Raymond Greenlaw and H. James Hoover. *Fundamentals of the Theory of Computation: Principles and Practice*. Morgan Kaufmann, 1998. ISBN: 978-1-55860-547-3. URL: https://www.sciencedirect.com/book/9781558605473/fundamentals-of-the-theory-of-computation-principles-and-practice (visited on 04/08/2026).

[7] Michael Hanus. *Modul Prinzipien von Programmiersprachen - Kapitel 21 Grundlagen*. 2020. URL: https://www.informatik.uni-kiel.de/~mh/lehre/pps20/vorlesung/kap21.pdf (visited on 04/08/2026).

[8] Itanium C++ ABI Committee. *Itanium C++ ABI: Exception Handling*. URL: https://itanium-cxx-abi.github.io/cxx-abi/abi-eh.html (visited on 04/07/2026).

[9] Ralf Jung et al. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages (POPL)*. Vol. 2. 2017. DOI: 10.1145/3158154. URL: https://dl.acm.org/doi/epdf/10.1145/3158154 (visited on 04/07/2026).

[10] Jeffrey Kegler. *Parsing: Top Down versus Bottom Up*. Nov. 2014. URL: https://blogs.perl.org/users/jeffrey_kegler/2014/11/parsing-top-down-versus-bottom-up.html (visited on 04/08/2026).

[11] klara. *Python ast transformation*. 2021. URL: https://klara-py.readthedocs.io/en/latest/cfg_ssa.html (visited on 04/08/2026).

[12] LLVM Project. *Exception Handling in LLVM*. URL: https://llvm.org/docs/ExceptionHandling.html (visited on 04/10/2026).

[13] LLVM Project. *JITLink and ORC's ObjectLinkingLayer*. URL: https://llvm.org/docs/JITLink.html (visited on 04/10/2026).

[14] Niko Matsakis. *LALRPOP on crates.io*. URL: https://crates.io/crates/lalrpop (visited on 04/08/2026).

[15] Niko Matsakis and LALRPOP Contributors. *LALRPOP Source Code*. URL: https://github.com/lalrpop/lalrpop (visited on 04/07/2026).

[16] Terence Parr, Sam Harwell, and ANTLR Contributors. *ANTLR4: ANother Tool for Language Recognition*. URL: https://github.com/antlr/antlr4 (visited on 04/07/2026).

[17] Massimiliano Poletto and Vivek Sarkar. "Linear Scan Register Allocation". In: *ACM Transactions on Programming Languages and Systems* (1999). DOI: 10.1145/330249.330250. URL: https://dl.acm.org/doi/abs/10.1145/330249.330250 (visited on 04/07/2026).

[18] Anoop Sarkar. *Static Single Assignment Form*. Nov. 15, 2020. URL: https://anoopsarkar.github.io/compilers-class/assets/lectures/ssaform2.pdf (visited on 04/08/2026).

[19] Jacob T. Schwartz. *Set Theory as a Language for Program Specification and Programming*. Tech. rep. Courant Institute of Mathematical Sciences, New York University, 1970. URL: https://softwarepreservation.computerhistory.org/SETL/ (visited on 04/07/2026).

[20] Keith Schwarz and Jinchao Ye. *Register Allocation*. 2012. URL: https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/17/Slides17.pdf (visited on 04/08/2026).

[21] Karl Stroetmann and Tom Herrmann. *SetlX - A Tutorial*. 2019. URL: https://download.randoom.org/setlX/tutorial.pdf (visited on 04/07/2026).

[22] Bjarne Stroustrup, Herb Sutter, and Gabriel Dos Reis. *A Brief Introduction to C++'s Model for Type- and Resource-Safety*. 2015. URL: https://www.stroustrup.com/resource-model.pdf (visited on 04/07/2026).

[23] Arjun Suresh. *LR Parsing Part 4: SLR, CLR, LALR and Summary*. Jan. 24, 2019. URL: https://gatecse.in/lr-parsing-part-4-slr-clr-lalr-and-summary/ (visited on 04/08/2026).

[24] The Rust Project Developers. *The Rust Programming Language*. URL: https://github.com/rust-lang/rust (visited on 04/07/2026).

[25] Gabriele Tomassetti. *A guide to Parsing: Algorithms and Terminology*. July 8, 2024. URL: https://tomassetti.me/guide-parsing-algorithms-terminology/ (visited on 04/08/2026).

[26] David Ungar. "Generation Scavenging: A Non-Disruptive High Performance Storage Reclamation Algorithm". In: *ACM SIGPLAN Notices* 19.5 (1984), pp. 157–167. DOI: 10.1145/390011.808929. URL: https://dl.acm.org/doi/10.1145/800020.808261 (visited on 04/10/2026).

# Glossary

**Abstract Syntax Tree**

An abstract syntax tree is a tree-shaped representation of source code that captures its semantic structure independently of the grammar rules used to parse it. Unlike a concrete syntax tree, its shape is not constrained by the grammar and can be designed to reflect the logical structure of the language more directly.

**Concrete Syntax Tree**

A concrete syntax tree or parse tree is a hierarchical tree representation of the syntactic structure of source code that directly reflects the grammar rules used to parse it.

**Control Flow Graph**

A control flow graph encodes all possible paths a program may take during execution. The code is divided into basic blocks, each of which becomes a node in the graph. If execution may transfer from one block to another via a jump or branch, a directed edge is added between the two.

**Directed Acyclic Graph**

A directed acyclic graph is graph made up of nodes that are interconnected through directed edges. A directed acyclic graph must not contain cycles, but must not necessarily be tree-shaped.

**Intermediate Representation**

An intermediate representation is a data structure used internally by a compiler or interpreter to represent a program between the source language and the target language. It is designed to be amenable to analysis and optimization.

**Landing Pad**

A landing pad is a compiler-generated block of code that serves as the entry point for the unwinder when it reaches a stack frame requiring attention during exception handling, ei-

ther because the frame contains a matching handler or because it has live owned objects that must be destroyed.

#### Resource Acquisition Is Initialization

Resource acquisition is initialization is a programming idiom in which a resource is acquired when an object is initialized and released when that object is destroyed, with the compiler inserting destructor calls at every exit point of a scope. It relies on ownership – the property that exactly one binding is responsible for a resource at any time – borrowing, which allows temporary access without transferring ownership, and lifetimes, which statically constrain how long a reference may be held.

#### Static Single Assignment Form

Static single assignment form imposes the constraint on an intermediate representation that each variable is assigned exactly once and that every variable is defined before it is used. This property simplifies many compiler optimizations such as constant propagation, copy propagation, and dead code elimination, as the single assignment guarantee makes the definition of every variable unambiguous.